

37. GPU 上位速度列探索

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	5
5.3 import 群の詳細	5
6 このファイルを読む前に必要な基礎知識	6
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	7
6.2 名前、代入、型、参照	7
6.3 関数、引数、戻り値、スコープ	7
6.4 module、package、import、console_scripts	8
6.5 例外、try/finally、with、資源解放	8
6.6 class、独自クラス、インスタンス、self、継承	8
6.7 先頭アンダースコア、__init__、名前付けの作法	9
6.8 list、dict、deque、NumPy 配列、pandas DataFrame	9
6.9 CLI、PowerShell、Bash、環境変数、終了コード	9
6.10 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明	9
6.11 Cross-Entropy Method を式と実装で理解する	10
6.12 天候、route、補間、時刻、単位	10
6.13 制御、状態、入力、モデル予測制御 MPC	10
6.14 warm start は何を保存し、どう効くか	11
6.15 単体試験、SILS、replay、観測可能性	11

7	関数・クラスを上から順に解説	12
7.1	L39 関数 <code>resolve</code>	12
7.2	L44 関数 <code>iso_utc</code>	12
7.3	L50 関数 <code>source_signature</code>	13
7.4	L62 関数 <code>sample_cem_noise</code>	14
7.5	L85 関数 <code>cem_should_stop</code>	15
7.6	L102 関数 <code>resolve_cuda_graph_enabled</code>	16
7.7	L109 関数 <code>build_distance_segments</code>	17
7.8	L133 関数 <code>kinetic_power_w</code>	18
7.9	L146 関数 <code>slew_limited_segment_kinematics</code>	19
7.10	L193 関数 <code>stationary_auxiliary_power_w</code>	21
7.11	L208 クラス <code>TensorMap2D</code>	22
7.12	L211 関数 <code>TensorMap2D.__init__</code>	23
7.13	L219 関数 <code>TensorMap2D.sample</code>	24
7.14	L236 クラス <code>TensorMap1D</code>	25
7.15	L237 関数 <code>TensorMap1D.__init__</code>	26
7.16	L244 関数 <code>TensorMap1D.sample</code>	27
7.17	L252 クラス <code>WeatherGrid</code>	27
7.18	L253 関数 <code>WeatherGrid.__init__</code>	29
7.19	L306 関数 <code>WeatherGrid.refine_time_grid</code>	31
7.20	L375 関数 <code>WeatherGrid._sample_matrix</code>	33
7.21	L392 関数 <code>WeatherGrid.sample</code>	35
7.22	L395 関数 <code>WeatherGrid.register_time_integral</code>	35
7.23	L405 関数 <code>WeatherGrid.sample_integral</code>	36
7.24	L428 関数 <code>WeatherGrid.register_soc_time_integral</code>	38
7.25	L454 関数 <code>WeatherGrid.sample_soc_integral</code>	39
7.26	L480 関数 <code>WeatherGrid.sample_soc_integral.interpolate</code>	41
7.27	L498 関数 <code>parse_args</code>	42
7.28	L554 関数 <code>main</code>	44
7.29	L912 関数 <code>main.pv_power</code>	48
7.30	L944 関数 <code>main.register_stationary_mode</code>	49
7.31	L1010 関数 <code>main.apply_wait</code>	51
7.32	L1091 関数 <code>main.evaluate</code>	53
7.33	L1097 関数 <code>main.evaluate.append_wait_trace</code>	57
7.34	CLI 引数	59
8	処理の流れ	59
9	このファイルの位置づけ	59

1 対象

項目	内容
ファイル	scripts/gpu_upper_policy_search.py
ソース SHA-256	a31c97605be77895407a9a6d9e1d153d0674f334f98c516f46c630f0ac9ad6e0
種別	Python
区分	planning research
役割	全レース distance-indexed speed policy を GPU 上で多段 coarse-to-fine に探索する。
起動文脈	本番前の warm start policy 生成側。

2 このファイルを読む理由

全レース distance-indexed speed policy を GPU 上で多段 coarse-to-fine に探索する。

- runtime MPC を置き換えるのではなく warm start policy 候補を作る。

3 呼び出し関係

3.1 どこから呼ばれるか

- GPUSbatch/shellcampaign

3.2 次に読むと理解が進むファイル

- scripts/validate_gpu_upper_policy_candidates.py
- scripts/run_upper_mesh_convergence.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/route_utils.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

GPU CEM proposer for a fine distance-domain upper speed policy.

This is deliberately a multi-fidelity proposer. Its vectorized energy model searches thousands of distance-indexed policies concurrently on CUDA. The selected policy must then be re-optimized and validated by “scripts/solar_sim.py”; the GPU surrogate is never treated as the production acceptance model.

4.1 代表コード断片

```
)

class TensorMap2D:
    """Bilinear CSV-map interpolation that stays on the selected torch device."""

    def __init__(self, csv_path: Path, device: torch.device):
```

```

frame = pd.read_csv(csv_path, index_col=0)
self.x = torch.as_tensor(frame.index.to_numpy(dtype=np.float32), device=device)
self.y = torch.as_tensor(frame.columns.to_numpy(dtype=np.float32), device=device)
self.z = torch.as_tensor(frame.to_numpy(dtype=np.float32), device=device)
if len(self.x) < 2 or len(self.y) < 2 or self.z.shape != (len(self.x), len(self.y)):
    raise ValueError(f"invalid 2-D map: {csv_path}")

def sample(self, x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    x_value = torch.minimum(torch.maximum(x, self.x[0]), self.x[-1]).contiguous()
    y_value = torch.minimum(torch.maximum(y, self.y[0]), self.y[-1]).contiguous()
    i1 = torch.clamp(torch.searchsorted(self.x, x_value), 1, len(self.x) - 1)
    j1 = torch.clamp(torch.searchsorted(self.y, y_value), 1, len(self.y) - 1)
    i0 = i1 - 1
    j0 = j1 - 1
    wx = (x_value - self.x[i0]) / torch.clamp(self.x[i1] - self.x[i0], min=1.0e-9)

```

5 主要構造の読み取り

主要クラスは TensorMap2D, TensorMap1D, WeatherGrid。主要関数は resolve, iso_utc, source_signature, sample_cem_noise, cem_should_stop, resolve_cuda_graph_enabled, build_distance_segments, kinetic_power_w。CLI 引数宣言は 20 件。

5.1 主要クラス・関数

行	種別	名前	補足
208	class	TensorMap2D	
236	class	TensorMap1D	
252	class	WeatherGrid	
39	function	resolve	args: profile, value
44	function	iso_utc	args: value
50	function	source_signature	args: paths
62	function	sample_cem_noise	args: population, dimensions
85	function	cem_should_stop	
102	function	resolve_cuda_graph_enabled	args: requested, integration_ds_km
109	function	build_distance_segments	args: race_km, integration_ds_km, stop_distances_km
133	function	kinetic_power_w	args: mass_kg, speed_ms, previous_speed_ms, dt_sec
146	function	slew_limited_segment_kinematics	args: previous_speed_ms, target_speed_ms, distance_km
193	function	stationary_auxiliary_power_w	args: irradiance_wm2
211	function	__init__	args: self, csv_path, device
219	function	sample	args: self, x, y
237	function	__init__	args: self, csv_path, device
244	function	sample	args: self, x
253	function	__init__	args: self, csv_path, start_utc, device
306	function	refine_time_grid	args: self, max_step_sec
375	function	_sample_matrix	args: self, matrix, t_sec, s_km

392	function	sample	args: self, name, t_sec, s_km
395	function	register_time_integral	args: self, name, rate
405	function	sample_integral	args: self, name, t_sec, s_km
428	function	register_soc_time_integral	args: self, name, soc_grid, rate
454	function	sample_soc_integral	args: self, name, t_sec, s_km, soc
480	function	interpolate	args: values, time_index
498	function	parse_args	
554	function	main	
912	function	pv_power	args: t_sec, s_km
944	function	register_stationary_mode	args: prefix, tilt_fraction
1010	function	apply_wait	args: t_sec, soc, tb_c, duration, s_km
1091	function	evaluate	args: policy
1097	function	append_wait_trace	args: label, segment_index, s_km, t_before, t_after, soc_before, soc_after, tb_before, tb_after
1443	function	evaluate_population	args: policy

5.2 ファイルを上から読んだときの定義順

行	内容
26	例外処理を伴う try ブロックを実行する。
32	ROOT に Path(__file__).resolve().parents[1] の結果を代入する。
33	条件 str(ROOT) not in sys.path を判定し、真なら内部処理を行う。
39	関数 resolve を定義する。
44	関数 iso_utc を定義する。
50	関数 source_signature を定義する。
62	関数 sample_cem_noise を定義する。
85	関数 cem_should_stop を定義する。
102	関数 resolve_cuda_graph_enabled を定義する。
109	関数 build_distance_segments を定義する。
133	関数 kinetic_power_w を定義する。
146	関数 slew_limited_segment_kinematics を定義する。
193	関数 stationary_auxiliary_power_w を定義する。
208	クラス TensorMap2D を定義する。
236	クラス TensorMap1D を定義する。
252	クラス WeatherGrid を定義する。
498	関数 parse_args を定義する。
554	関数 main を定義する。
1693	条件 __name__ == '__main__' を判定し、真なら内部処理を行う。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
---	----------	-------------

10	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
12	<code>importargparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L498, L499, L509, L515, L521, L544。
13	<code>importhashlib</code>	snapshot ID や入力資産の digest を作るため。このファイル内での主な使用位置は L51。
14	<code>importjson</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L58, L891, L1451, L1556, L1605, L1686, L1689。
15	<code>importmath</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L314, L342, L678, L1179。
16	<code>importsys</code>	sys モジュールを利用するため。このファイル内での主な使用位置は L33, L34。
17	<code>fromdatetimeimportdatetime, timedelta,timezone</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L44, L46, L47, L253, L734, L742, L751, L752, ...。
18	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L32, L39, L40, L50, L211, L237, L253, L500, ...。
19	<code>fromzoneinfoimportZoneInfo</code>	zoneinfo から ZoneInfo を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L691。
21	<code>importnumpyasnp</code>	数値配列、clip、補間、統計計算を行うため。このファイル内での主な使用位置は L113, L115, L116, L117, L120, L121, L127, L128, ...。
22	<code>importpandasaspd</code>	CSV や時刻列の読み込み・整形・再サンプリングに使うため。このファイル内での主な使用位置は L212, L238, L254, L255, L257, L259, L265, L267, ...。
23	<code>importtorch</code>	torch モジュールを利用するため。このファイル内での主な使用位置は L66, L68, L75, L80, L81, L82, L135, L136, ...。
24	<code>importyaml</code>	profile、stop、schedule、summary YAML を読み書きするため。このファイル内での主な使用位置は L565, L578, L686。
27	<code>fromtorch.utils. tensorboardimportSummaryWriter</code>	torch.utils.tensorboard から SummaryWriter を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L29, L904。
36	<code>frommpc_solarcar.route_ utilsimportaverage_profile_ segments</code>	route_utils.py から average_profile_segments を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/route_utils.py。このファイル内での主な使用位置は L626, L629。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や ‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport -> main()を呼び

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体を作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、`__init__`、名前付けの作法

‘self. step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘`__init__`’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N,)’、候補集団なら ‘(population, N)’となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.9 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。‘argparse’は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS_DOMAIN_ID、RMW_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

6.10 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は cost 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 seed や大域探索と組み合わせる。success が False でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた `sampling` と局所最適化を組み合わせる大域最適化法である。有限 Cartesian grid の全列挙は、その grid 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- SciPy 公式: [scipy.optimize.shgo](#)

6.11 Cross-Entropy Method を式と実装で理解する

CEM は候補を生成する確率分布を持ち、良かった elite 候補から分布を更新する反復的な確率最適化である。このリポジトリの `upper_solver.py` は各制御点速度を独立正規分布で生成し、上下限へ clip する。

$$u_i^{(j)} \sim \mathcal{N}(\mu_i^{(g)}, (\sigma_i^{(g)})^2), \quad u_i^{(j)} \leftarrow \text{clip}(u_i^{(j)}, l_i, h_i)$$
$$\mu_i^{(g+1)} = \frac{1}{K} \sum_{j \in \mathcal{E}_g} u_i^{(j)}, \quad \sigma_i^{(g+1)} = \max \left(\text{Std}_{j \in \mathcal{E}_g} u_i^{(j)}, 0.05(h_i - l_i) \right)$$

ここで $\mathcal{E}_g$ は cost が小さい上位 K 候補である。平均は良い領域へ移り、標準偏差は探索幅を表す。標準偏差の下限は探索が完全に潰れることを避ける。

現行 `hybrid_bounded_minimize` は、`deterministic seed` を評価し、上位候補を L-BFGS-B で局所 refine し、設定と seed 間不一致に応じて CEM を実行し、最後に再度局所 refine する。したがって CEM 単独ではなく hybrid solver である。

CEM で落とした候補を永久保存しないこと自体は通常最適化として自然だが、off-nominal 状態からの再利用には別の policy library 設計が必要である。状態を無制限に全組合せ保存する代わりに、SoC、進捗、時刻、予報誤差、停止状態などの scenario を設計し、近傍 policy を検索して MPC で再最適化する。

根拠資料

- Rubinstein and Kroese: [The Cross-Entropy Method](#)
- SciPy 公式: [scipy.optimize.minimize](#)

6.12 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.13 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- [SciPy 公式: `scipy.optimize.minimize`](#)

6.14 warm start は何を保存し、どう効くか

warm start は前回または offline 探索で得た入力系列を、次の最適化の初期候補として渡すことである。答えを固定するのではなく、探索開始点と候補 library の一員を与える。

$$u_i^{0,\text{new}} = \text{interp} \left(s_i^{\text{new}}; s_j^{\text{old}}, u_j^{*,\text{old}} \right)$$

距離基準計画では、現在より後ろの旧制御点を捨て、新しい絶対距離制御点へ補間して shift する。初期状態や天候が変われば cost 評価は新条件で行われるので、warm start が不適切でも他 seed、CEM、安全 fallback が補う設計が必要である。

warm start の効き方は、局所法なら収束先と反復数、CEM なら初期 mean または候補 pool、receding horizon なら前回解の時間・距離 shift として現れる。どの位置に渡しているかを呼出引数まで追う。

根拠資料

- [SciPy 公式: `scipy.optimize.minimize`](#)
- [Rubinstein and Kroese: The Cross-Entropy Method](#)

6.15 単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、historical replay は実測入力への再現性、本番 preflight は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、solver 成否、候補数、計算時間、fallback、source revision、profile hash を同じ run ID で記録する。

非同期不具合は最終値だけでは追えない。topic 周期、message age、callback 所要時間、queue 遅延、Node 生存、publisher 数を同時に観測する。

根拠資料

- [ROS 2 Humble 公式: `rqt\_graph`](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 公式: Executors](#)

7 関数・クラスを上から順に解説

7.1 L39 関数 resolve

- 行範囲: L39–L41
- 所属: module 直下
- 定義: `resolve(profile:Path,value:str)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_absolute`, `resolve`, `str`
- 戻り値の要点: `path if path.is_absolute() else (profile.parent / path).resolve()`
- 主なローカル代入名: `path`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `path` に `Path(str(value))` の結果を代入する。
2. `path if path.is_absolute() else (profile.parent / path).resolve()` を返す。

代表コード断片

```
def resolve(profile: Path, value: str) -> Path:
    path = Path(str(value))
    return path if path.is_absolute() else (profile.parent / path).resolve()
```

7.2 L44 関数 iso_utc

- 行範囲: L44–L47
- 所属: module 直下
- 定義: `iso_utc(value:str)->datetime`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `astimezone`, `fromisoformat`, `replace`, `str`
- 戻り値の要点: `parsed.replace(tzinfo=timezone.utc) if parsed.tzinfo is None else parsed.astimezone(timezone.utc)`
- 主なローカル代入名: `parsed`, `text`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. text に `str(value).replace('Z', '+00:00')` の結果を代入する。
2. parsed に `datetime.fromisoformat(text)` の結果を代入する。
3. `parsed.replace(tzinfo=datetime.timezone.utc)` if `parsed.tzinfo is None` else `parsed.astimezone(datetime.timezone.utc)` を返す。

代表コード断片

```
def iso_utc(value: str) -> datetime:
    text = str(value).replace("Z", "+00:00")
    parsed = datetime.fromisoformat(text)
    return parsed.replace(tzinfo=datetime.timezone.utc) if parsed.tzinfo is None else parsed.
        astimezone(datetime.timezone.utc)
```

7.3 L50 関数 source_signature

- 行範囲: L50–L59
- 所属: module 直下
- 定義: `source_signature(paths:list[Path],*,settings:dict)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dumps`, `encode`, `hexdigest`, `iter`, `open`, `read`, `resolve`, `sha256`, `str`, `update`
- 戻り値の要点: `digest.hexdigest()`
- 主なローカル代入名: `chunk`, `digest`, `handle`, `path`, `resolved`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 2、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. digest に hashlib.sha256() の結果を代入する。
2. paths を順に走査し、各要素を path に入れて処理する。
3. resolved に path.resolve() の結果を代入する。
4. digest.update(...) を実行する。
5. with 文で resolved.open('rb') を管理しながら処理する。
6. iter(lambda: handle.read(1024 * 1024), b'') を順に走査し、各要素を chunk に入れて処理する。
7. digest.update(...) を実行する。
8. digest.update(...) を実行する。
9. digest.hexdigest() を返す。

代表コード断片

```
def source_signature(paths: list[Path], *, settings: dict) -> str:
    digest = hashlib.sha256()
    for path in paths:
        resolved = path.resolve()
        digest.update(str(resolved).encode("utf-8"))
        with resolved.open("rb") as handle:
            for chunk in iter(lambda: handle.read(1024 * 1024), b''):
                digest.update(chunk)
    digest.update(json.dumps(settings, sort_keys=True, separators=(",", ":")).encode("utf-8"))
    return digest.hexdigest()
```

7.4 L62 関数 sample_cem_noise

- 行範囲: L62–L82
- 所属: module 直下
- 定義: `sample_cem_noise(population:int,dimensions:int,*,device:torch.device,antithetic:bool)->torch.Tensor`
- docstring: Sample CEM perturbations, reserving the first row for the current mean.
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `cat`, `int`, `randn`, `zero_`, `zeros`
- 戻り値の要点: `torch.cat((torch.zeros((1, dimensions), device=device), paired), dim=0) / noise`
- 主なローカル代入名: `dimensions`, `noise`, `pair_count`, `paired`, `population`, `positive`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('population and dimensions must be positive')`
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. population に `int(population)` の結果を代入する。
2. dimensions に `int(dimensions)` の結果を代入する。
3. 条件 `population <= 0 or dimensions <= 0` を判定し、真なら内部処理を行う。
4. `ValueError('population and dimensions must be positive')` を送出する。
5. 条件 `not antithetic or population == 1` を判定し、真なら内部処理を行う。
6. noise に `torch.randn((population, dimensions), device=device)` の結果を代入する。
7. `noise[0].zero_()` を実行する。
8. noise を返す。
9. pair_count に `(population - 1 + 1) // 2` の結果を代入する。
10. positive に `torch.randn((pair_count, dimensions), device=device)` の結果を代入する。
11. paired に `torch.cat((positive, -positive), dim=0)[:population - 1]` の結果を代入する。
12. `torch.cat((torch.zeros((1, dimensions), device=device), paired), dim=0)` を返す。

代表コード断片

```
def sample_cem_noise(
    population: int,
    dimensions: int,
    *,
    device: torch.device,
    antithetic: bool,
) -> torch.Tensor:
    """Sample CEM perturbations, reserving the first row for the current mean."""
    population = int(population)
    dimensions = int(dimensions)
    if population <= 0 or dimensions <= 0:
        raise ValueError("population and dimensions must be positive")
    if not antithetic or population == 1:
        noise = torch.randn((population, dimensions), device=device)
        noise[0].zero_()
        return noise

    pair_count = (population - 1 + 1) // 2
    positive = torch.randn((pair_count, dimensions), device=device)
    paired = torch.cat((positive, -positive), dim=0)[:population - 1]
    return torch.cat((torch.zeros((1, dimensions), device=device), paired), dim=0)
```

7.5 L85 関数 cem_should_stop

- 行範囲: L85–L99
- 所属: module 直下
- 定義: `cem_should_stop(*, generation_completed:int, stagnant_generations:int, mean_std_kmh:float, patience:int, min_generations:int, max_std_kmh:float) -> bool`
- docstring: Return true only after both objective and sampling spread have converged.
- このブロックが直接呼ぶ主な関数・メソッド: float, int, max
- 戻り値の要点: `float(max_std_kmh) <= 0.0 or float(mean_std_kmh) <= float(max_std_kmh) / False / False`

- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出す例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `int(patience) <= 0 or int(generation_completed) + 1 < max(0, int(min_generations))` を判定し、真なら内部処理を行う。
2. `False` を返す。
3. 条件 `int(stagnant_generations) < int(patience)` を判定し、真なら内部処理を行う。
4. `False` を返す。
5. `float(max_std_kmh) <= 0.0 or float(mean_std_kmh) <= float(max_std_kmh)` を返す。

代表コード断片

```
def cem_should_stop(
    *,
    generation_completed: int,
    stagnant_generations: int,
    mean_std_kmh: float,
    patience: int,
    min_generations: int,
    max_std_kmh: float,
) -> bool:
    """Return true only after both objective and sampling spread have converged."""
    if int(patience) <= 0 or int(generation_completed) + 1 < max(0, int(min_generations)):
        return False
    if int(stagnant_generations) < int(patience):
        return False
    return float(max_std_kmh) <= 0.0 or float(mean_std_kmh) <= float(max_std_kmh)
```

7.6 L102 関数 `resolve_cuda_graph_enabled`

- 行範囲: L102–L106
- 所属: module 直下
- 定義: `resolve_cuda_graph_enabled(requested:bool|None,integration_ds_km:float)->bool`
- docstring: Auto-enable graphs only for the coarse rollout shape covered by the benchmark.
- このブロックが直接呼ぶ主な関数・メソッド: `bool`, `float`
- 戻り値の要点: `float(integration_ds_km) >= 5.0 / bool(requested)`
- 主なローカル代入名: 特になし。

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `requested is not None` を判定し、真なら内部処理を行う。
2. `bool(requested)` を返す。
3. `float(integration_ds_km) >= 5.0` を返す。

代表コード断片

```
def resolve_cuda_graph_enabled(requested: bool | None, integration_ds_km: float) -> bool:
    """Auto-enable graphs only for the coarse rollout shape covered by the benchmark."""
    if requested is not None:
        return bool(requested)
    return float(integration_ds_km) >= 5.0
```

7.7 L109 関数 `build_distance_segments`

- 行範囲: L109–L130
- 所属: module 直下
- 定義: `build_distance_segments(race_km:float,integration_ds_km:float,stop_distances_km:list[float])->tuple[np.ndarray,np.ndarray,np.ndarray]`
- docstring: Build integration segments with every physical stop as an exact boundary.
- このブロックが直接呼ぶ主な関数・メソッド: `arange`, `asarray`, `astype`, `concatenate`, `diff`, `unique`
- 戻り値の要点: `(boundaries[:-1].astype(np.float32), np.diff(boundaries).astype(np.float32), boundaries)`
- 主なローカル代入名: `base_boundaries`, `boundaries`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `base_boundaries` に `np.arange(0.0, race_km, integration_ds_km, dtype=np.float64)` の結果を代入する。
2. `boundaries` に `np.unique(np.concatenate((base_boundaries, np.asarray(stop_distances_km, dtype=np.float64), np.asarray([race_km], dtype=np.float64))))` の結果を代入する。
3. `boundaries` に `boundaries[(boundaries >= 0.0) & (boundaries <= race_km)]` の結果を代入する。
4. `(boundaries[:-1].astype(np.float32), np.diff(boundaries).astype(np.float32), boundaries)` を返す。

代表コード断片

```
def build_distance_segments(
    race_km: float,
    integration_ds_km: float,
    stop_distances_km: list[float],
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Build integration segments with every physical stop as an exact boundary."""
    base_boundaries = np.arange(0.0, race_km, integration_ds_km, dtype=np.float64)
    boundaries = np.unique(
        np.concatenate(
            (
                base_boundaries,
                np.asarray(stop_distances_km, dtype=np.float64),
                np.asarray([race_km], dtype=np.float64),
            )
        )
    )
    boundaries = boundaries[(boundaries >= 0.0) & (boundaries <= race_km)]
    return (
        boundaries[:-1].astype(np.float32),
        np.diff(boundaries).astype(np.float32),
        boundaries,
    )
```

7.8 L133 関数 `kinetic_power_w`

- 行範囲: L133–L143
- 所属: module 直下
- 定義: `kinetic_power_w(mass_kg:float, speed_ms:torch.Tensor, previous_speed_ms:torch.Tensor, dt_sec:torch.Tensor)->torch.Tensor`
- docstring: Signed wheel power whose interval integral equals the kinetic-energy change.
- このブロックが直接呼ぶ主な関数・メソッド: `clamp`, `float`
- 戻り値の要点: `delta_energy_j / torch.clamp(dt_sec, min=1e-06)`
- 主なローカル代入名: `delta_energy_j`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `delta_energy_j` に `0.5 * float(mass_kg) * (speed_ms * speed_ms - previous_speed_ms * previous_speed_ms)` の結果を代入する。
2. `delta_energy_j / torch.clamp(dt_sec, min=1e-06)` を返す。

代表コード断片

```
def kinetic_power_w(
    mass_kg: float,
    speed_ms: torch.Tensor,
    previous_speed_ms: torch.Tensor,
    dt_sec: torch.Tensor,
) -> torch.Tensor:
    """Signed wheel power whose interval integral equals the kinetic-energy change."""
    delta_energy_j = 0.5 * float(mass_kg) * (
        speed_ms * speed_ms - previous_speed_ms * previous_speed_ms
    )
    return delta_energy_j / torch.clamp(dt_sec, min=1.0e-6)
```

7.9 L146 関数 `slew_limited_segment_kinematics`

- 行範囲: L146–L190
- 所属: module 直下
- 定義: `slew_limited_segment_kinematics(previous_speed_ms:torch.Tensor,target_speed_ms:torch.Tensor,distance_km:float,*,accel_limit_kmhps:float,decel_limit_kmhps:float)->tuple[torch.Tensor,torch.Tensor,torch.Tensor]`
- docstring: Return distance-average speed, end speed, and time for a slew-limited segment.
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `clamp`, `float`, `full_like`, `max`, `sqrt`, `where`, `zeros_like`
- 戻り値の要点: (`average_speed_ms`, `end_speed_ms`, `duration_sec`)
- 主なローカル代入名: `accel_ms2`, `accelerating`, `average_speed_ms`, `cruise_distance_m`, `cruise_time_sec`, `decel_ms2`, `distance_m`, `distance_to_target_m`, `duration_sec`, `end_speed_ms`, `limited_end_speed_ms`, `limited_end_sq`, `ramp_distance_m`, `ramp_time_sec`, `rate_ms2`, `reaches_target`, `signed_rate_ms2`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. distance_m に $\max(0.0, \text{float}(\text{distance_km}) * 1000.0)$ の結果を代入する。
2. accel_ms2 に $\max(1e-06, \text{float}(\text{accel_limit_kmhps}) / 3.6)$ の結果を代入する。
3. decel_ms2 に $\max(1e-06, \text{float}(\text{decel_limit_kmhps}) / 3.6)$ の結果を代入する。
4. accelerating に $\text{target_speed_ms} \geq \text{previous_speed_ms}$ の結果を代入する。
5. rate_ms2 に $\text{torch.where}(\text{accelerating}, \text{torch.full_like}(\text{target_speed_ms}, \text{accel_ms2}), \text{torch.full_like}(\text{target_speed_ms}, \text{decel_ms2}))$ の結果を代入する。
6. signed_rate_ms2 に $\text{torch.where}(\text{accelerating}, \text{rate_ms2}, -\text{rate_ms2})$ の結果を代入する。
7. distance_to_target_m に $\text{torch.abs}(\text{target_speed_ms} * \text{target_speed_ms} - \text{previous_speed_ms} * \text{previous_speed_ms}) / (2.0 * \text{rate_ms2})$ の結果を代入する。
8. reaches_target に $\text{distance_to_target_m} \leq \text{distance_m} + 1e-07$ の結果を代入する。
9. limited_end_sq に $\text{torch.clamp}(\text{previous_speed_ms} * \text{previous_speed_ms} + 2.0 * \text{signed_rate_ms2} * \text{distance_m}, \text{min}=0.0)$ の結果を代入する。
10. limited_end_speed_ms に $\text{torch.sqrt}(\text{limited_end_sq})$ の結果を代入する。
11. end_speed_ms に $\text{torch.where}(\text{reaches_target}, \text{target_speed_ms}, \text{limited_end_speed_ms})$ の結果を代入する。
12. ramp_distance_m に $\text{torch.where}(\text{reaches_target}, \text{distance_to_target_m}, \text{torch.full_like}(\text{distance_to_target_m}, \text{distance_m}))$ の結果を代入する。
13. ramp_time_sec に $\text{torch.abs}(\text{end_speed_ms} - \text{previous_speed_ms}) / \text{rate_ms2}$ の結果を代入する。
14. cruise_distance_m に $\text{torch.clamp}(\text{distance_m} - \text{ramp_distance_m}, \text{min}=0.0)$ の結果を代入する。
15. cruise_time_sec に $\text{torch.where}(\text{reaches_target}, \text{cruise_distance_m} / \text{torch.clamp}(\text{target_speed_ms}, \text{min}=0.001), \text{torch.zeros_like}(\text{cruise_distance_m}))$ の結果を代入する。
16. duration_sec に $\text{torch.clamp}(\text{ramp_time_sec} + \text{cruise_time_sec}, \text{min}=1e-06)$ の結果を代入する。
17. average_speed_ms に $\text{torch.full_like}(\text{duration_sec}, \text{distance_m}) / \text{duration_sec}$ の結果を代入する。
18. (average_speed_ms, end_speed_ms, duration_sec) を返す。

代表コード断片

```
def slew_limited_segment_kinematics(
    previous_speed_ms: torch.Tensor,
    target_speed_ms: torch.Tensor,
    distance_km: float,
    *,
    accel_limit_kmhps: float,
    decel_limit_kmhps: float,
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
    """Return distance-average speed, end speed, and time for a slew-limited segment."""
    distance_m = max(0.0, float(distance_km) * 1000.0)
    accel_ms2 = max(1.0e-6, float(accel_limit_kmhps) / 3.6)
    decel_ms2 = max(1.0e-6, float(decel_limit_kmhps) / 3.6)
    accelerating = target_speed_ms >= previous_speed_ms
    rate_ms2 = torch.where(
        accelerating,
        torch.full_like(target_speed_ms, accel_ms2),
        torch.full_like(target_speed_ms, decel_ms2),
    )
    signed_rate_ms2 = torch.where(accelerating, rate_ms2, -rate_ms2)
    distance_to_target_m = torch.abs(
```

```

        target_speed_ms * target_speed_ms
        - previous_speed_ms * previous_speed_ms
    ) / (2.0 * rate_ms2)
    reaches_target = distance_to_target_m <= distance_m + 1.0e-7
    limited_end_sq = torch.clamp(
        previous_speed_ms * previous_speed_ms + 2.0 * signed_rate_ms2 * distance_m,
        min=0.0,
    )
    limited_end_speed_ms = torch.sqrt(limited_end_sq)
    end_speed_ms = torch.where(reaches_target, target_speed_ms, limited_end_speed_ms)
    ramp_distance_m = torch.where(
        reaches_target,
        distance_to_target_m,
        torch.full_like(distance_to_target_m, distance_m),
    )
...

```

7.10 L193 関数 stationary_auxiliary_power_w

- 行範囲: L193–L205
- 所属: module 直下
- 定義: `stationary_auxiliary_power_w(irradiance_wm2:torch.Tensor,*, day_power_w:float,night_power_w:float,night_threshold_wm2:float)->torch.Tensor`
- docstring: Match the production model’s stopped day/night auxiliary-power rule.
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `full_like`, `max`, `where`
- 戻り値の要点: `torch.where(irradiance_wm2 > max(0.0, float(night_threshold_wm2)), torch.full_like(irradiance_wm2, max(0.0, float(day_power_w))), torch.full_like(irradiance_wm2, max(0.0, float(night_power_w))))`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `torch.where(irradiance_wm2 > max(0.0, float(night_threshold_wm2)), torch.full_like(irradiance_wm2, max(0.0, float(day_power_w))), torch.full_like(irradiance_wm2, max(0.0, float(night_power_w))))` を返す。

代表コード断片

```

def stationary_auxiliary_power_w(
    irradiance_wm2: torch.Tensor,
    *,
    day_power_w: float,
    night_power_w: float,
    night_threshold_wm2: float,
) -> torch.Tensor:

```

```

"""Match the production model's stopped day/night auxiliary-power rule."""
return torch.where(
    irradiance_wm2 > max(0.0, float(night_threshold_wm2)),
    torch.full_like(irradiance_wm2, max(0.0, float(day_power_w))),
    torch.full_like(irradiance_wm2, max(0.0, float(night_power_w))),
)

```

7.11 L208 クラス TensorMap2D

- 行範囲: L208–L233
- 所属: module 直下
- 定義: TensorMap2D(bases=None)
- docstring: Bilinear CSV-map interpolation that stays on the selected torch device.
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。

上から順の処理

1. docstring または説明文字列を置く。
2. 関数 __init__ を定義する。
3. 関数 sample を定義する。

代表コード断片

```

class TensorMap2D:
    """Bilinear CSV-map interpolation that stays on the selected torch device."""

    def __init__(self, csv_path: Path, device: torch.device):
        frame = pd.read_csv(csv_path, index_col=0)
        self.x = torch.as_tensor(frame.index.to_numpy(dtype=np.float32), device=device)
        self.y = torch.as_tensor(frame.columns.to_numpy(dtype=np.float32), device=device)
        self.z = torch.as_tensor(frame.to_numpy(dtype=np.float32), device=device)
        if len(self.x) < 2 or len(self.y) < 2 or self.z.shape != (len(self.x), len(self.y)):
            raise ValueError(f"invalid 2-D map: {csv_path}")

    def sample(self, x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        x_value = torch.minimum(torch.maximum(x, self.x[0]), self.x[-1]).contiguous()
        y_value = torch.minimum(torch.maximum(y, self.y[0]), self.y[-1]).contiguous()
        i1 = torch.clamp(torch.searchsorted(self.x, x_value), 1, len(self.x) - 1)
        j1 = torch.clamp(torch.searchsorted(self.y, y_value), 1, len(self.y) - 1)

```

```

i0 = i1 - 1
j0 = j1 - 1
wx = (x_value - self.x[i0]) / torch.clamp(self.x[i1] - self.x[i0], min=1.0e-9)
wy = (y_value - self.y[j0]) / torch.clamp(self.y[j1] - self.y[j0], min=1.0e-9)
return (
    (1.0 - wx) * (1.0 - wy) * self.z[i0, j0]
    + wx * (1.0 - wy) * self.z[i1, j0]
    + (1.0 - wx) * wy * self.z[i0, j1]
    + wx * wy * self.z[i1, j1]
)

```

7.12 L211 関数 TensorMap2D.__init__

- 行範囲: L211–L217
- 所属: TensorMap2D
- 定義: `__init__(self, csv_path: Path, device: torch.device)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `as_tensor`, `len`, `read_csv`, `to_numpy`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `frame`
- 読み取る主なインスタンス属性: `self.x`, `self.y`, `self.z`
- 更新する主なインスタンス属性: `self.x`, `self.y`, `self.z`
- 明示的に送出する例外: `ValueError(f'invalid 2-D map: {csv_path}')`
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `frame` に `pd.read_csv(csv_path, index_col=0)` の結果を代入する。
2. `self.x` に `torch.as_tensor(frame.index.to_numpy(dtype=np.float32), device=device)` の結果を代入する。
3. `self.y` に `torch.as_tensor(frame.columns.to_numpy(dtype=np.float32), device=device)` の結果を代入する。
4. `self.z` に `torch.as_tensor(frame.to_numpy(dtype=np.float32), device=device)` の結果を代入する。
5. 条件 `len(self.x) < 2 or len(self.y) < 2 or self.z.shape != (len(self.x), len(self.y))` を判定し、真なら内部処理を行う。
6. `ValueError(f'invalid 2-D map: {csv_path}')` を送出する。

代表コード断片

```

def __init__(self, csv_path: Path, device: torch.device):
    frame = pd.read_csv(csv_path, index_col=0)
    self.x = torch.as_tensor(frame.index.to_numpy(dtype=np.float32), device=device)
    self.y = torch.as_tensor(frame.columns.to_numpy(dtype=np.float32), device=device)
    self.z = torch.as_tensor(frame.to_numpy(dtype=np.float32), device=device)
    if len(self.x) < 2 or len(self.y) < 2 or self.z.shape != (len(self.x), len(self.y)):
        raise ValueError(f"invalid 2-D map: {csv_path}")

```

7.13 L219 関数 TensorMap2D.sample

- 行範囲: L219–L233
- 所属: TensorMap2D
- 定義: `sample(self, x: torch.Tensor, y: torch.Tensor) -> torch.Tensor`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clamp`, `contiguous`, `len`, `maximum`, `minimum`, `searchsorted`
- 戻り値の要点: $(1.0 - wx) * (1.0 - wy) * \text{self.z}[i0, j0] + wx * (1.0 - wy) * \text{self.z}[i1, j0] + (1.0 - wx) * wy * \text{self.z}[i0, j1] + wx * wy * \text{self.z}[i1, j1]$
- 主なローカル代入名: `i0`, `i1`, `j0`, `j1`, `wx`, `wy`, `x_value`, `y_value`
- 読み取る主なインスタンス属性: `self.x`, `self.y`, `self.z`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `x_value` に `torch.minimum(torch.maximum(x, self.x[0]), self.x[-1]).contiguous()` の結果を代入する。
2. `y_value` に `torch.minimum(torch.maximum(y, self.y[0]), self.y[-1]).contiguous()` の結果を代入する。
3. `i1` に `torch.clamp(torch.searchsorted(self.x, x_value), 1, len(self.x) - 1)` の結果を代入する。
4. `j1` に `torch.clamp(torch.searchsorted(self.y, y_value), 1, len(self.y) - 1)` の結果を代入する。
5. `i0` に `i1 - 1` の結果を代入する。
6. `j0` に `j1 - 1` の結果を代入する。
7. `wx` に $(x_value - \text{self.x}[i0]) / \text{torch.clamp}(\text{self.x}[i1] - \text{self.x}[i0], \text{min}=1\text{e-}09)$ の結果を代入する。
8. `wy` に $(y_value - \text{self.y}[j0]) / \text{torch.clamp}(\text{self.y}[j1] - \text{self.y}[j0], \text{min}=1\text{e-}09)$ の結果を代入する。
9. $(1.0 - wx) * (1.0 - wy) * \text{self.z}[i0, j0] + wx * (1.0 - wy) * \text{self.z}[i1, j0] + (1.0 - wx) * wy * \text{self.z}[i0, j1] + wx * wy * \text{self.z}[i1, j1]$ を返す。

代表コード断片

```
def sample(self, x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    x_value = torch.minimum(torch.maximum(x, self.x[0]), self.x[-1]).contiguous()
    y_value = torch.minimum(torch.maximum(y, self.y[0]), self.y[-1]).contiguous()
    i1 = torch.clamp(torch.searchsorted(self.x, x_value), 1, len(self.x) - 1)
    j1 = torch.clamp(torch.searchsorted(self.y, y_value), 1, len(self.y) - 1)
    i0 = i1 - 1
    j0 = j1 - 1
    wx = (x_value - self.x[i0]) / torch.clamp(self.x[i1] - self.x[i0], min=1.0e-9)
    wy = (y_value - self.y[j0]) / torch.clamp(self.y[j1] - self.y[j0], min=1.0e-9)
    return (
        (1.0 - wx) * (1.0 - wy) * self.z[i0, j0]
        + wx * (1.0 - wy) * self.z[i1, j0]
        + (1.0 - wx) * wy * self.z[i0, j1]
        + wx * wy * self.z[i1, j1]
    )
```

7.14 L236 クラス TensorMap1D

- 行範囲: L236–L249
- 所属: module 直下
- 定義: TensorMap1D(bases=None)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。

上から順の処理

1. 関数 __init__ を定義する。
2. 関数 sample を定義する。

代表コード断片

```
class TensorMap1D:
    def __init__(self, csv_path: Path, device: torch.device):
        frame = pd.read_csv(csv_path)
        if frame.shape[1] < 2 or len(frame) < 2:
            raise ValueError(f"invalid 1-D map: {csv_path}")
        self.x = torch.as_tensor(frame.iloc[:, 0].to_numpy(dtype=np.float32), device=device)
        self.y = torch.as_tensor(frame.iloc[:, 1].to_numpy(dtype=np.float32), device=device)
```

```
def sample(self, x: torch.Tensor) -> torch.Tensor:
    value = torch.minimum(torch.maximum(x, self.x[0]), self.x[-1]).contiguous()
    i1 = torch.clamp(torch.searchsorted(self.x, value), 1, len(self.x) - 1)
    i0 = i1 - 1
    weight = (value - self.x[i0]) / torch.clamp(self.x[i1] - self.x[i0], min=1.0e-9)
    return self.y[i0] * (1.0 - weight) + self.y[i1] * weight
```

7.15 L237 関数 TensorMap1D.__init__

- 行範囲: L237–L242
- 所属: TensorMap1D
- 定義: `__init__(self, csv_path: Path, device: torch.device)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `as_tensor`, `len`, `read_csv`, `to_numpy`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `frame`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.x`, `self.y`
- 明示的に送出する例外: `ValueError(f'invalid 1-D map: {csv_path}')`
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `frame` に `pd.read_csv(csv_path)` の結果を代入する。
2. 条件 `frame.shape[1] < 2 or len(frame) < 2` を判定し、真なら内部処理を行う。
3. `ValueError(f'invalid 1-D map: {csv_path}')` を送出する。
4. `self.x` に `torch.as_tensor(frame.iloc[:, 0].to_numpy(dtype=np.float32), device=device)` の結果を代入する。
5. `self.y` に `torch.as_tensor(frame.iloc[:, 1].to_numpy(dtype=np.float32), device=device)` の結果を代入する。

代表コード断片

```
def __init__(self, csv_path: Path, device: torch.device):
    frame = pd.read_csv(csv_path)
    if frame.shape[1] < 2 or len(frame) < 2:
        raise ValueError(f"invalid 1-D map: {csv_path}")
    self.x = torch.as_tensor(frame.iloc[:, 0].to_numpy(dtype=np.float32), device=device)
    self.y = torch.as_tensor(frame.iloc[:, 1].to_numpy(dtype=np.float32), device=device)
```

7.16 L244 関数 TensorMap1D.sample

- 行範囲: L244–L249
- 所属: TensorMap1D
- 定義: `sample(self, x: torch.Tensor) -> torch.Tensor`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clamp`, `contiguous`, `len`, `maximum`, `minimum`, `searchsorted`
- 戻り値の要点: `self.y[i0] * (1.0 - weight) + self.y[i1] * weight`
- 主なローカル代入名: `i0`, `i1`, `value`, `weight`
- 読み取る主なインスタンス属性: `self.x`, `self.y`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `value` に `torch.minimum(torch.maximum(x, self.x[0]), self.x[-1]).contiguous()` の結果を代入する。
2. `i1` に `torch.clamp(torch.searchsorted(self.x, value), 1, len(self.x) - 1)` の結果を代入する。
3. `i0` に `i1 - 1` の結果を代入する。
4. `weight` に `(value - self.x[i0]) / torch.clamp(self.x[i1] - self.x[i0], min=1e-09)` の結果を代入する。
5. `self.y[i0] * (1.0 - weight) + self.y[i1] * weight` を返す。

代表コード断片

```
def sample(self, x: torch.Tensor) -> torch.Tensor:
    value = torch.minimum(torch.maximum(x, self.x[0]), self.x[-1]).contiguous()
    i1 = torch.clamp(torch.searchsorted(self.x, value), 1, len(self.x) - 1)
    i0 = i1 - 1
    weight = (value - self.x[i0]) / torch.clamp(self.x[i1] - self.x[i0], min=1.0e-9)
    return self.y[i0] * (1.0 - weight) + self.y[i1] * weight
```

7.17 L252 クラス WeatherGrid

- 行範囲: L252–L495
- 所属: module 直下
- 定義: `WeatherGrid(bases=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。

- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `refine_time_grid` を定義する。
3. 関数 `_sample_matrix` を定義する。
4. 関数 `sample` を定義する。
5. 関数 `register_time_integral` を定義する。
6. 関数 `sample_integral` を定義する。
7. 関数 `register_soc_time_integral` を定義する。
8. 関数 `sample_soc_integral` を定義する。

代表コード断片

```
class WeatherGrid:
    def __init__(self, csv_path: Path, start_utc: datetime, device: torch.device):
        frame = pd.read_csv(csv_path)
        source_text = frame.get("weather_source", pd.Series(dtype=str)).astype(str).str.lower()
        applied_gain = (
            pd.to_numeric(frame["headwind_gain_applied"], errors="coerce")
            if "headwind_gain_applied" in frame.columns
            else pd.Series(np.nan, index=frame.index)
        )
        self.headwind_already_scaled = bool(
            applied_gain.notna().any()
            or source_text.str.contains("headwind_scaled|corrected.*headwind", regex=True).any()
        )
        frame["time"] = pd.to_datetime(frame["time"], utc=True, errors="coerce")
        frame = frame.dropna(subset=["time", "s_km"])
        times = pd.Index(frame["time"].drop_duplicates().sort_values())
        distances = np.array(sorted(pd.to_numeric(frame["s_km"], errors="coerce").dropna().unique()), dtype=np.float32)
        self.time_sec = torch.as_tensor(
            ((times - pd.Timestamp(start_utc)).total_seconds()).to_numpy(dtype=np.float32),
            device=device
        )
        self.distance_km = torch.as_tensor(distances, device=device)
```

```

self.distance_np = distances
self.values = {}
self.integrals = {}
self.integral_rates = {}
self.soc_integrals = {}
self.soc_integral_rates = {}
self.soc_integral_grids = {}
weather_fields = (
    ("GHI", 0.0, True),
    ("Tamb_C", 30.0, True),
    ("Tcell_C", 35.0, True),
    ("headwind_ms", 0.0, True),
    ("POA_drive", 0.0, False),
    ("POA_stop_ideal", 0.0, False),
...

```

7.18 L253 関数 WeatherGrid.__init__

- 行範囲: L253–L304
- 所属: WeatherGrid
- 定義: `__init__(self, csv_path:Path, start_utc:datetime, device:torch.device)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Index`, `Series`, `Timestamp`, `any`, `array`, `as_tensor`, `astype`, `bfill`, `bool`, `contains`, `drop_duplicates`, `dropna`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `applied_gain`, `default`, `distances`, `frame`, `matrix`, `name`, `required`, `source_text`, `times`, `weather_fields`
- 読み取る主なインスタンス属性: `self.values`
- 更新する主なインスタンス属性: `self.distance_km`, `self.distance_np`, `self.headwind_already_scaled`, `self.integral_rates`, `self.integrals`, `self.soc_integral_grids`, `self.soc_integral_rates`, `self.soc_integrals`, `self.time_sec`, `self.values`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `frame` に `pd.read_csv(csv_path)` の結果を代入する。
2. `source_text` に `frame.get('weather_source', pd.Series(dtype=str)).astype(str).str.lower()` の結果を代入する。
3. `applied_gain` に `pd.to_numeric(frame['headwind_gain_applied'], errors='coerce')` if `'headwind_gain_applied'` in `frame.columns` else `pd.Series(np.nan, index=frame.index)` の結果を代入する。
4. `self.headwind_already_scaled` に `bool(applied_gain.notna().any() or source_text.str.contains('headwind_scaled|corrected.', regex=True).any())` の結果を代入する。

5. `frame['time']` に `pd.to_datetime(frame['time'], utc=True, errors='coerce')` の結果を代入する。
6. `frame` に `frame.dropna(subset=['time', 's_km'])` の結果を代入する。
7. `times` に `pd.Index(frame['time'].drop_duplicates().sort_values())` の結果を代入する。
8. `distances` に `np.array(sorted(pd.to_numeric(frame['s_km'], errors='coerce').dropna().unique()), dtype=np.float32)` の結果を代入する。
9. `self.time_sec` に `torch.as_tensor((times - pd.Timestamp(start_utc)).total_seconds().to_numpy(dtype=np.float32), device=device)` の結果を代入する。
10. `self.distance_km` に `torch.as_tensor(distances, device=device)` の結果を代入する。
11. `self.distance_np` に `distances` の結果を代入する。
12. `self.values` に `{}` の結果を代入する。
13. `self.integrals` に `{}` の結果を代入する。
14. `self.integral_rates` に `{}` の結果を代入する。
15. `self.soc_integrals` に `{}` の結果を代入する。
16. `self.soc_integral_rates` に `{}` の結果を代入する。
17. `self.soc_integral_grids` に `{}` の結果を代入する。
18. `weather_fields` に `((('GHI', 0.0, True), ('Tamb_C', 30.0, True), ('Tcell_C', 35.0, True), ('headwind_ms', 0.0, True), ('POA_drive', 0.0, False), ('POA_stop_ideal', 0.0, False), ('Tcell_drive_C', 35.0, False), ('Tcell_stop_ideal_C', 35.0, False)))` の結果を代入する。
19. `weather_fields` を順に走査し、各要素を `(name, default, required)` に入れて処理する。
20. 条件 `name not in frame.columns` を判定し、真なら内部処理を行う。
21. 条件 `not required` を判定し、真なら内部処理を行う。
22. `Continue` 文を実行する。
23. `matrix` に `np.full((len(times), len(distances)), default, dtype=np.float32)` の結果を代入する。
24. 上の条件が偽の場合:
25. `matrix` に `frame.pivot_table(index='time', columns='s_km', values=name, aggfunc='last').reindex(index=times, columns=distances).interpolate(axis=0, limit_direction='both').interpolate(axis=1, limit_direction='both').ffill().bfill().ffill(copy=True)` の結果を代入する。
26. `self.values[name]` に `torch.as_tensor(np.array(matrix, copy=True), device=device)` の結果を代入する。

代表コード断片

```
def __init__(self, csv_path: Path, start_utc: datetime, device: torch.device):
    frame = pd.read_csv(csv_path)
    source_text = frame.get("weather_source", pd.Series(dtype=str)).astype(str).str.
lower()
    applied_gain = (
        pd.to_numeric(frame["headwind_gain_applied"], errors="coerce")
        if "headwind_gain_applied" in frame.columns
        else pd.Series(np.nan, index=frame.index)
    )
    self.headwind_already_scaled = bool(
        applied_gain.notna().any()
        or source_text.str.contains("headwind_scaled|corrected.*headwind", regex=True).
any()
    )
```

```

        frame["time"] = pd.to_datetime(frame["time"], utc=True, errors="coerce")
        frame = frame.dropna(subset=["time", "s_km"])
        times = pd.Index(frame["time"].drop_duplicates().sort_values())
        distances = np.array(sorted(pd.to_numeric(frame["s_km"], errors="coerce").dropna().
unique()), dtype=np.float32)
        self.time_sec = torch.as_tensor(
            ((times - pd.Timestamp(start_utc)).total_seconds()).to_numpy(dtype=np.float32),
device=device
        )
        self.distance_km = torch.as_tensor(distances, device=device)
        self.distance_np = distances
        self.values = {}
        self.integrals = {}
        self.integral_rates = {}
        self.soc_integrals = {}
        self.soc_integral_rates = {}
        self.soc_integral_grids = {}
        weather_fields = (
            ("GHI", 0.0, True),
            ("Tamb_C", 30.0, True),
            ("Tcell_C", 35.0, True),
            ("headwind_ms", 0.0, True),
            ("POA_drive", 0.0, False),
            ("POA_stop_ideal", 0.0, False),
            ("Tcell_drive_C", 35.0, False),
        )
...

```

7.19 L306 関数 WeatherGrid.refine_time_grid

- 行範囲: L306–L373
- 所属: WeatherGrid
- 定義: `refine_time_grid(self, max_step_sec: float) -> dict[str, float | int]`
- docstring: Linearly densify weather before applying nonlinear component models.
Interpolating hourly PV power or an hourly day/night switch is not equivalent to interpolating irradiance first. The production model does the latter, so the GPU proposer must use the same operation order.
- このブロックが直接呼ぶ主な関数・メソッド: `RuntimeError`, `ValueError`, `any`, `append`, `arange`, `as_tensor`, `asarray`, `astype`, `ceil`, `clamp`, `cpu`, `detach`
- 戻り値の要点: `{'source_points': int(len(source_np)), 'refined_points': int(len(refined_np)), 'source_max_step_sec': source_max_step, 'refined_max_step_sec': refined_max_step} / {'source_points': int(len(source_np)), 'refined_points': int(len(source_np)), 'source_max_step_sec': 0.0, 'refined_max_step_sec': 0.0} / {'source_points': int(len(source_np)), 'refined_points': int(len(source_np)), 'source_max_step_sec': source_max_step, 'refined_max_step_sec': source_max_step}`
- 主なローカル代入名: `left`, `lower`, `matrix`, `name`, `refined_max_step`, `refined_np`, `refined_parts`, `refined_times`, `requested_step`, `right`, `source_max_step`, `source_np`, `source_steps`, `source_times`, `subdivisions`, `upper`, `weight`
- 読み取る主なインスタンス属性: `self.integrals`, `self.soc_integrals`, `self.time_sec`, `self.values`
- 更新する主なインスタンス属性: `self.time_sec`, `self.values`
- 明示的に送出する例外: `RuntimeError('weather must be refined before registering integrals')`, `ValueError('weather refinement step must be positive')`, `ValueError('weather timestamps must be strictly increasing')`
- 制御構造の規模: 条件分岐 5、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `requested_step` に `float(max_step_sec)` の結果を代入する。
2. 条件 `not math.isfinite(requested_step) or requested_step <= 0.0` を判定し、真なら内部処理を行う。
3. `ValueError('weather refinement step must be positive')` を送出する。
4. 条件 `self.integrals or self.soc_integrals` を判定し、真なら内部処理を行う。
5. `RuntimeError('weather must be refined before registering integrals')` を送出する。
6. `source_times` に `self.time_sec` の結果を代入する。
7. `source_np` に `source_times.detach().cpu().numpy().astype(float)` の結果を代入する。
8. 条件 `len(source_np) < 2` を判定し、真なら内部処理を行う。
9. `{'source_points': int(len(source_np)), 'refined_points': int(len(source_np)), 'source_max_step_sec': 0.0, 'refined_max_step_sec': 0.0}` を返す。
10. `source_steps` に `np.diff(source_np)` の結果を代入する。
11. 条件 `np.any(source_steps <= 0.0)` を判定し、真なら内部処理を行う。
12. `ValueError('weather timestamps must be strictly increasing')` を送出する。
13. `source_max_step` に `float(np.max(source_steps))` の結果を代入する。
14. 条件 `source_max_step <= requested_step + 1e-06` を判定し、真なら内部処理を行う。
15. `{'source_points': int(len(source_np)), 'refined_points': int(len(source_np)), 'source_max_step_sec': source_max_step, 'refined_max_step_sec': source_max_step}` を返す。
16. `refined_parts` に `[]` の結果を代入する。
17. `zip(source_np[:-1], source_np[1:])` を順に走査し、各要素を `(left, right)` に入れて処理する。
18. `subdivisions` に `max(1, int(math.ceil((right - left) / requested_step)))` の結果を代入する。
19. `refined_parts.extend(...)` を実行する。
20. `refined_np` に `np.append(np.asarray(refined_parts, dtype=np.float32), np.float32(source_np[-1]))` の結果を代入する。
21. `refined_times` に `torch.as_tensor(refined_np, dtype=source_times.dtype, device=source_times.device)` の結果を代入する。
22. `upper` に `torch.clamp(torch.searchsorted(source_times, refined_times), 1, len(source_times) - 1)` の結果を代入する。
23. `lower` に `upper - 1` の結果を代入する。
24. `weight` に `((refined_times - source_times[lower]) / torch.clamp(source_times[upper] - source_times[lower], min=1.0))[ :, None]` の結果を代入する。

25. self.values に {name: matrix[lower] * (1.0 - weight) + matrix[upper] * weight for name, matrix in self.values.items()} の結果を代入する。
26. self.time_sec に refined_times の結果を代入する。
27. refined_max_step に float(np.max(np.diff(refined_np.astype(float)))) の結果を代入する。
28. {'source_points': int(len(source_np)), 'refined_points': int(len(refined_np)), 'source_max_step_sec': source_max_step, 'refined_max_step_sec': refined_max_step} を返す。

代表コード断片

```
def refine_time_grid(self, max_step_sec: float) -> dict[str, float | int]:
    """Linearly densify weather before applying nonlinear component models.

    Interpolating hourly PV power or an hourly day/night switch is not
    equivalent to interpolating irradiance first. The production model
    does the latter, so the GPU proposer must use the same operation order.
    """
    requested_step = float(max_step_sec)
    if not math.isfinite(requested_step) or requested_step <= 0.0:
        raise ValueError("weather refinement step must be positive")
    if self.integrals or self.soc_integrals:
        raise RuntimeError("weather must be refined before registering integrals")

    source_times = self.time_sec
    source_np = source_times.detach().cpu().numpy().astype(float)
    if len(source_np) < 2:
        return {
            "source_points": int(len(source_np)),
            "refined_points": int(len(source_np)),
            "source_max_step_sec": 0.0,
            "refined_max_step_sec": 0.0,
        }
    source_steps = np.diff(source_np)
    if np.any(source_steps <= 0.0):
        raise ValueError("weather timestamps must be strictly increasing")
    source_max_step = float(np.max(source_steps))
    if source_max_step <= requested_step + 1.0e-6:
        return {
            "source_points": int(len(source_np)),
            "refined_points": int(len(source_np)),
            "source_max_step_sec": source_max_step,
            "refined_max_step_sec": source_max_step,
        }

    refined_parts = []
    ...
```

7.20 L375 関数 WeatherGrid._sample_matrix

- 行範囲: L375–L390
- 所属: WeatherGrid
- 定義: `_sample_matrix(self, matrix: torch.Tensor, t_sec: torch.Tensor, s_km: float) -> torch.Tensor`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: clamp, clip, float, int, len, max, maximum, minimum, searchsorted

- 戻り値の要点: $\text{low} * (1.0 - \text{wt}) + \text{high} * \text{wt}$
- 主なローカル代入名: `high, i0, i1, j0_int, j1_int, low, s, t, tg, ws, wt`
- 読み取る主なインスタンス属性: `self.distance_np, self.time_sec`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `tg` に `self.time_sec` の結果を代入する。
2. `t` に `torch.minimum(torch.maximum(t_sec, tg[0]), tg[-1])` の結果を代入する。
3. `s` に `float(np.clip(s_km, float(self.distance_np[0]), float(self.distance_np[-1])))` の結果を代入する。
4. `i1` に `torch.clamp(torch.searchsorted(tg, t), 1, len(tg) - 1)` の結果を代入する。
5. `i0` に `i1 - 1` の結果を代入する。
6. `j1_int` に `int(np.clip(np.searchsorted(self.distance_np, s), 1, len(self.distance_np) - 1))` の結果を代入する。
7. `j0_int` に `j1_int - 1` の結果を代入する。
8. `wt` に `(t - tg[i0]) / torch.clamp(tg[i1] - tg[i0], min=1.0)` の結果を代入する。
9. `ws` に `(s - float(self.distance_np[j0_int])) / max(float(self.distance_np[j1_int] - self.distance_np[j0_int]), 1e-06)` の結果を代入する。
10. `low` に `matrix[i0, j0_int] * (1.0 - ws) + matrix[i0, j1_int] * ws` の結果を代入する。
11. `high` に `matrix[i1, j0_int] * (1.0 - ws) + matrix[i1, j1_int] * ws` の結果を代入する。
12. `low * (1.0 - wt) + high * wt` を返す。

代表コード断片

```
def _sample_matrix(self, matrix: torch.Tensor, t_sec: torch.Tensor, s_km: float) ->
    torch.Tensor:
    tg = self.time_sec
    t = torch.minimum(torch.maximum(t_sec, tg[0]), tg[-1])
    s = float(np.clip(s_km, float(self.distance_np[0]), float(self.distance_np[-1])))
    i1 = torch.clamp(torch.searchsorted(tg, t), 1, len(tg) - 1)
    i0 = i1 - 1
    j1_int = int(np.clip(np.searchsorted(self.distance_np, s), 1, len(self.distance_np)
- 1))
    j0_int = j1_int - 1
    wt = (t - tg[i0]) / torch.clamp(tg[i1] - tg[i0], min=1.0)
    ws = (s - float(self.distance_np[j0_int])) / max(
        float(self.distance_np[j1_int] - self.distance_np[j0_int]),
        1.0e-6,
    )
    low = matrix[i0, j0_int] * (1.0 - ws) + matrix[i0, j1_int] * ws
    high = matrix[i1, j0_int] * (1.0 - ws) + matrix[i1, j1_int] * ws
    return low * (1.0 - wt) + high * wt
```

7.21 L392 関数 WeatherGrid.sample

- 行範囲: L392–L393
- 所属: WeatherGrid
- 定義: `sample(self, name:str, t_sec:torch.Tensor, s_km:float) -> torch.Tensor`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_sample_matrix`
- 戻り値の要点: `self._sample_matrix(self.values[name], t_sec, s_km)`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._sample_matrix`, `self.values`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self._sample_matrix(self.values[name], t_sec, s_km)` を返す。

代表コード断片

```
def sample(self, name: str, t_sec: torch.Tensor, s_km: float) -> torch.Tensor:
    return self._sample_matrix(self.values[name], t_sec, s_km)
```

7.22 L395 関数 WeatherGrid.register_time_integral

- 行範囲: L395–L403
- 所属: WeatherGrid
- 定義: `register_time_integral(self, name:str, rate:torch.Tensor) -> None`
- docstring: Register the trapezoidal time integral of a W-valued weather-grid field.
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `cat`, `cumsum`, `iter`, `next`, `values`, `zeros`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `dt`, `increments`, `zero`
- 読み取る主なインスタンス属性: `self.integral_rates`, `self.integrals`, `self.time_sec`, `self.values`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('integrated weather field must match the weather grid shape')`
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `rate.shape != next(iter(self.values.values())).shape` を判定し、真なら内部処理を行う。
2. `ValueError('integrated weather field must match the weather grid shape')` を送出する。
3. `dt` に `self.time_sec[1:] - self.time_sec[:-1]` の結果を代入する。
4. `increments` に `0.5 * (rate[1:] + rate[:-1]) * dt[:, None]` の結果を代入する。
5. `zero` に `torch.zeros((1, rate.shape[1]), dtype=rate.dtype, device=rate.device)` の結果を代入する。
6. `self.integrals[name]` に `torch.cat((zero, torch.cumsum(increments, dim=0)), dim=0)` の結果を代入する。
7. `self.integral_rates[name]` に `rate` の結果を代入する。

代表コード断片

```
def register_time_integral(self, name: str, rate: torch.Tensor) -> None:
    """Register the trapezoidal time integral of a W-valued weather-grid field."""
    if rate.shape != next(iter(self.values.values())).shape:
        raise ValueError("integrated weather field must match the weather grid shape")
    dt = self.time_sec[1:] - self.time_sec[:-1]
    increments = 0.5 * (rate[1:] + rate[:-1]) * dt[:, None]
    zero = torch.zeros((1, rate.shape[1]), dtype=rate.dtype, device=rate.device)
    self.integrals[name] = torch.cat((zero, torch.cumsum(increments, dim=0)), dim=0)
    self.integral_rates[name] = rate
```

7.23 L405 関数 `WeatherGrid.sample_integral`

- 行範囲: L405–L426
- 所属: `WeatherGrid`
- 定義: `sample_integral(self, name: str, t_sec: torch.Tensor, s_km: float) -> torch.Tensor`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clamp`, `clip`, `float`, `int`, `len`, `max`, `maximum`, `minimum`, `searchsorted`
- 戻り値の要点: `cumulative_0 + partial`
- 主なローカル代入名: `cumulative`, `cumulative_0`, `i0`, `i1`, `interval`, `j0_int`, `j1_int`, `partial`, `rate`, `rate_0`, `rate_1`, `s`, `t`, `tg`, `ws`, `wt`
- 読み取る主なインスタンス属性: `self.distance_np`, `self.integral_rates`, `self.integrals`, `self.time_sec`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `tg` に `self.time_sec` の結果を代入する。
2. `t` に `torch.minimum(torch.maximum(t_sec, tg[0]), tg[-1])` の結果を代入する。
3. `s` に `float(np.clip(s_km, float(self.distance_np[0]), float(self.distance_np[-1])))` の結果を代入する。
4. `i1` に `torch.clamp(torch.searchsorted(tg, t), 1, len(tg) - 1)` の結果を代入する。
5. `i0` に `i1 - 1` の結果を代入する。
6. `j1_int` に `int(np.clip(np.searchsorted(self.distance_np, s), 1, len(self.distance_np) - 1))` の結果を代入する。
7. `j0_int` に `j1_int - 1` の結果を代入する。
8. `interval` に `torch.clamp(tg[i1] - tg[i0], min=1.0)` の結果を代入する。
9. `wt` に `(t - tg[i0]) / interval` の結果を代入する。
10. `ws` に `(s - float(self.distance_np[j0_int])) / max(float(self.distance_np[j1_int] - self.distance_np[j0_int]), 1e-06)` の結果を代入する。
11. `cumulative` に `self.integrals[name]` の結果を代入する。
12. `rate` に `self.integral_rates[name]` の結果を代入する。
13. `cumulative_0` に `cumulative[i0, j0_int] * (1.0 - ws) + cumulative[i0, j1_int] * ws` の結果を代入する。
14. `rate_0` に `rate[i0, j0_int] * (1.0 - ws) + rate[i0, j1_int] * ws` の結果を代入する。
15. `rate_1` に `rate[i1, j0_int] * (1.0 - ws) + rate[i1, j1_int] * ws` の結果を代入する。
16. `partial` に `(t - tg[i0]) * (rate_0 + 0.5 * (rate_1 - rate_0) * wt)` の結果を代入する。
17. `cumulative_0 + partial` を返す。

代表コード断片

```
def sample_integral(self, name: str, t_sec: torch.Tensor, s_km: float) -> torch.Tensor:
    tg = self.time_sec
    t = torch.minimum(torch.maximum(t_sec, tg[0]), tg[-1])
    s = float(np.clip(s_km, float(self.distance_np[0]), float(self.distance_np[-1])))
    i1 = torch.clamp(torch.searchsorted(tg, t), 1, len(tg) - 1)
    i0 = i1 - 1
    j1_int = int(np.clip(np.searchsorted(self.distance_np, s), 1, len(self.distance_np)
- 1))
    j0_int = j1_int - 1
    interval = torch.clamp(tg[i1] - tg[i0], min=1.0)
    wt = (t - tg[i0]) / interval
    ws = (s - float(self.distance_np[j0_int])) / max(
        float(self.distance_np[j1_int] - self.distance_np[j0_int]),
        1.0e-6,
    )

    cumulative = self.integrals[name]
    rate = self.integral_rates[name]
    cumulative_0 = cumulative[i0, j0_int] * (1.0 - ws) + cumulative[i0, j1_int] * ws
```

```

rate_0 = rate[i0, j0_int] * (1.0 - ws) + rate[i0, j1_int] * ws
rate_1 = rate[i1, j0_int] * (1.0 - ws) + rate[i1, j1_int] * ws
partial = (t - tg[i0]) * (rate_0 + 0.5 * (rate_1 - rate_0) * wt)
return cumulative_0 + partial

```

7.24 L428 関数 WeatherGrid.register_soc_time_integral

- 行範囲: L428–L452
- 所属: WeatherGrid
- 定義: `register_soc_time_integral(self, name:str, soc_grid:torch.Tensor, rate:torch.Tensor)->None`
- docstring: Register a time integral whose rate also varies over battery SoC.
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `cat`, `cumsum`, `iter`, `len`, `next`, `tuple`, `values`, `zeros`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `dt`, `expected`, `increments`, `zero`
- 読み取る主なインスタンス属性: `self.soc_integral_grids`, `self.soc_integral_rates`, `self.soc_integrals`, `self.time_sec`, `self.values`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('SoC grid length does not match the integrated field')`, `ValueError('SoC-integrated field must have shape [soc, time, distance]')`
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `expected` に `next(iter(self.values.values())).shape` の結果を代入する。
2. 条件 `rate.ndim != 3 or tuple(rate.shape[1:]) != tuple(expected)` を判定し、真なら内部処理を行う。
3. `ValueError('SoC-integrated field must have shape [soc, time, distance]')` を送出する。
4. 条件 `rate.shape[0] != len(soc_grid)` を判定し、真なら内部処理を行う。
5. `ValueError('SoC grid length does not match the integrated field')` を送出する。
6. `dt` に `self.time_sec[1:] - self.time_sec[:-1]` の結果を代入する。
7. `increments` に `0.5 * (rate[:, 1:] + rate[:, :-1]) * dt[None, :, None]` の結果を代入する。
8. `zero` に `torch.zeros((rate.shape[0], 1, rate.shape[2]), dtype=rate.dtype, device=rate.device)` の結果を代入する。
9. `self.soc_integrals[name]` に `torch.cat((zero, torch.cumsum(increments, dim=1)), dim=1)` の結果を代入する。
10. `self.soc_integral_rates[name]` に `rate` の結果を代入する。
11. `self.soc_integral_grids[name]` に `soc_grid` の結果を代入する。

代表コード断片

```
def register_soc_time_integral(
    self,
    name: str,
    soc_grid: torch.Tensor,
    rate: torch.Tensor,
) -> None:
    """Register a time integral whose rate also varies over battery SoC."""
    expected = next(iter(self.values.values())).shape
    if rate.ndim != 3 or tuple(rate.shape[1:]) != tuple(expected):
        raise ValueError("SoC-integrated field must have shape [soc, time, distance]")
    if rate.shape[0] != len(soc_grid):
        raise ValueError("SoC grid length does not match the integrated field")
    dt = self.time_sec[1:] - self.time_sec[:-1]
    increments = 0.5 * (rate[:, 1:] + rate[:, :-1]) * dt[None, :, None]
    zero = torch.zeros(
        (rate.shape[0], 1, rate.shape[2]),
        dtype=rate.dtype,
        device=rate.device,
    )
    self.soc_integrals[name] = torch.cat(
        (zero, torch.cumsum(increments, dim=1)),
        dim=1,
    )
    self.soc_integral_rates[name] = rate
    self.soc_integral_grids[name] = soc_grid
```

7.25 L454 関数 WeatherGrid.sample_soc_integral

- 行範囲: L454–L495
- 所属: WeatherGrid
- 定義: `sample_soc_integral(self, name: str, t_sec: torch.Tensor, s_km: float, soc: torch.Tensor) -> torch.Tensor`
- docstring: Trilinearly sample a cumulative [SoC,time,distance] integral.
- このブロックが直接呼ぶ主な関数・メソッド: `clamp`, `clip`, `contiguous`, `float`, `int`, `interpolate`, `len`, `max`, `maximum`, `minimum`, `searchsorted`
- 戻り値の要点: $\text{cumulative_0} + \text{partial} / \text{low_soc} * (1.0 - \text{wz}) + \text{high_soc} * \text{wz}$
- 主なローカル代入名: `cumulative`, `cumulative_0`, `high_soc`, `high_soc_high_s`, `high_soc_low_s`, `i0`, `i1`, `j0`, `j1`, `k0`, `k1`, `low_soc`, `low_soc_high_s`, `low_soc_low_s`, `partial`, `rate`, `rate_0`, `rate_1`, `s`, `t`
- 読み取る主なインスタンス属性: `self.distance_np`, `self.soc_integral_grids`, `self.soc_integral_rates`, `self.soc_integrals`, `self.time_sec`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. tg に self.time_sec の結果を代入する。
2. zg に self.soc_integral_grids[name] の結果を代入する。
3. t に torch.minimum(torch.maximum(t_sec, tg[0]), tg[-1]).contiguous() の結果を代入する。
4. z に torch.minimum(torch.maximum(soc, zg[0]), zg[-1]).contiguous() の結果を代入する。
5. s に float(np.clip(s_km, float(self.distance_np[0]), float(self.distance_np[-1]))) の結果を代入する。
6. i1 に torch.clamp(torch.searchsorted(tg, t), 1, len(tg) - 1) の結果を代入する。
7. i0 に i1 - 1 の結果を代入する。
8. k1 に torch.clamp(torch.searchsorted(zg, z), 1, len(zg) - 1) の結果を代入する。
9. k0 に k1 - 1 の結果を代入する。
10. j1 に int(np.clip(np.searchsorted(self.distance_np, s), 1, len(self.distance_np) - 1)) の結果を代入する。
11. j0 に j1 - 1 の結果を代入する。
12. wt に (t - tg[i0]) / torch.clamp(tg[i1] - tg[i0], min=1.0) の結果を代入する。
13. wz に (z - zg[k0]) / torch.clamp(zg[k1] - zg[k0], min=1e-09) の結果を代入する。
14. ws に (s - float(self.distance_np[j0])) / max(float(self.distance_np[j1] - self.distance_np[j0]), 1e-06) の結果を代入する。
15. 関数 interpolate を定義する。
16. cumulative に self.soc_integrals[name] の結果を代入する。
17. rate に self.soc_integral_rates[name] の結果を代入する。
18. cumulative_0 に interpolate(cumulative, i0) の結果を代入する。
19. rate_0 に interpolate(rate, i0) の結果を代入する。
20. rate_1 に interpolate(rate, i1) の結果を代入する。
21. partial に (t - tg[i0]) * (rate_0 + 0.5 * (rate_1 - rate_0) * wt) の結果を代入する。
22. cumulative_0 + partial を返す。

代表コード断片

```
def sample_soc_integral(
    self,
    name: str,
    t_sec: torch.Tensor,
    s_km: float,
    soc: torch.Tensor,
) -> torch.Tensor:
    """Trilinearly sample a cumulative [SoC,time,distance] integral."""
    tg = self.time_sec
    zg = self.soc_integral_grids[name]
    t = torch.minimum(torch.maximum(t_sec, tg[0]), tg[-1]).contiguous()
    z = torch.minimum(torch.maximum(soc, zg[0]), zg[-1]).contiguous()
    s = float(np.clip(s_km, float(self.distance_np[0]), float(self.distance_np[-1])))
    i1 = torch.clamp(torch.searchsorted(tg, t), 1, len(tg) - 1)
    i0 = i1 - 1
    k1 = torch.clamp(torch.searchsorted(zg, z), 1, len(zg) - 1)
    k0 = k1 - 1
```

```

        j1 = int(np.clip(np.searchsorted(self.distance_np, s), 1, len(self.distance_np) - 1)
    )
    j0 = j1 - 1
    wt = (t - tg[i0]) / torch.clamp(tg[i1] - tg[i0], min=1.0)
    wz = (z - zg[k0]) / torch.clamp(zg[k1] - zg[k0], min=1.0e-9)
    ws = (s - float(self.distance_np[j0])) / max(
        float(self.distance_np[j1] - self.distance_np[j0]),
        1.0e-6,
    )

    def interpolate(values: torch.Tensor, time_index: torch.Tensor) -> torch.Tensor:
        low_soc_low_s = values[k0, time_index, j0]
        low_soc_high_s = values[k0, time_index, j1]
        high_soc_low_s = values[k1, time_index, j0]
        high_soc_high_s = values[k1, time_index, j1]
        low_soc = low_soc_low_s * (1.0 - ws) + low_soc_high_s * ws
        high_soc = high_soc_low_s * (1.0 - ws) + high_soc_high_s * ws
        return low_soc * (1.0 - wz) + high_soc * wz

...

```

7.26 L480 関数 WeatherGrid.sample_soc_integral.interpolate

- 行範囲: L480–L487
- 所属: WeatherGrid.sample_soc_integral
- 定義: interpolate(values:torch.Tensor,time_index:torch.Tensor)->torch.Tensor
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: low_soc * (1.0 - wz) + high_soc * wz
- 主なローカル代入名: high_soc, high_soc_high_s, high_soc_low_s, low_soc, low_soc_high_s, low_soc_low_s
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. low_soc_low_s に values[k0, time_index, j0] の結果を代入する。
2. low_soc_high_s に values[k0, time_index, j1] の結果を代入する。
3. high_soc_low_s に values[k1, time_index, j0] の結果を代入する。
4. high_soc_high_s に values[k1, time_index, j1] の結果を代入する。
5. low_soc に low_soc_low_s * (1.0 - ws) + low_soc_high_s * ws の結果を代入する。
6. high_soc に high_soc_low_s * (1.0 - ws) + high_soc_high_s * ws の結果を代入する。
7. low_soc * (1.0 - wz) + high_soc * wz を返す。

代表コード断片

```
def interpolate(values: torch.Tensor, time_index: torch.Tensor) -> torch.Tensor:
    low_soc_low_s = values[k0, time_index, j0]
    low_soc_high_s = values[k0, time_index, j1]
    high_soc_low_s = values[k1, time_index, j0]
    high_soc_high_s = values[k1, time_index, j1]
    low_soc = low_soc_low_s * (1.0 - ws) + low_soc_high_s * ws
    high_soc = high_soc_low_s * (1.0 - ws) + high_soc_high_s * ws
    return low_soc * (1.0 - wz) + high_soc * wz
```

7.27 L498 関数 parse_args

- 行範囲: L498–L551
- 所属: module 直下
- 定義: `parse_args()->argparse.Namespace`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `add_argument`, `parse_args`
- 戻り値の要点: `parser.parse_args()`
- 主なローカル代入名: `parser`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `parser` に `argparse.ArgumentParser(description=__doc__)` の結果を代入する。
2. `parser.add_argument(...)` を実行する。
3. `parser.add_argument(...)` を実行する。
4. `parser.add_argument(...)` を実行する。
5. `parser.add_argument(...)` を実行する。
6. `parser.add_argument(...)` を実行する。
7. `parser.add_argument(...)` を実行する。
8. `parser.add_argument(...)` を実行する。
9. `parser.add_argument(...)` を実行する。
10. `parser.add_argument(...)` を実行する。
11. `parser.add_argument(...)` を実行する。

12. `parser.add_argument(...)` を実行する。
13. `parser.add_argument(...)` を実行する。
14. `parser.add_argument(...)` を実行する。
15. `parser.add_argument(...)` を実行する。
16. `parser.add_argument(...)` を実行する。
17. `parser.add_argument(...)` を実行する。
18. `parser.add_argument(...)` を実行する。
19. `parser.add_argument(...)` を実行する。
20. `parser.add_argument(...)` を実行する。
21. `parser.add_argument(...)` を実行する。
22. `parser.parse_args()` を返す。

代表コード断片

```
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--profile", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--integration-ds-km", type=float, default=1.0)
    parser.add_argument("--control-ds-km", type=float, default=25.0)
    parser.add_argument("--population", type=int, default=4096)
    parser.add_argument("--elite", type=int, default=128)
    parser.add_argument("--generations", type=int, default=80)
    parser.add_argument("--seed", type=int, default=20260715)
    parser.add_argument("--checkpoint-every", type=int, default=10)
    parser.add_argument("--resume", action=argparse.BooleanOptionalAction, default=True)
    parser.add_argument("--tensorboard-dir", type=Path)
    parser.add_argument("--initial-policy", type=Path)
    parser.add_argument("--initial-std-kmh", type=float, default=4.0)
    parser.add_argument(
        "--antithetic-sampling",
        action=argparse.BooleanOptionalAction,
        default=False,
        help="Use paired positive/negative perturbations to reduce sampling variance.",
    )
    parser.add_argument(
        "--cuda-graph",
        action=argparse.BooleanOptionalAction,
        default=None,
        help=(
            "Capture the fixed-shape surrogate rollout as one replayable CUDA graph. "
            "The default enables it only for integration grids of 5 km or coarser."
        ),
    )
    parser.add_argument(
        "--early-stop-patience",
        type=int,
        default=0,
        help="Stop after this many generations without a significant improvement; zero disables.",
    )
    ...
```

7.28 L554 関数 main

- 行範囲: L554–L1690
- 所属: module 直下
- 定義: `main()->int`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `CUDAGraph`, `DataFrame`, `Path`, `RuntimeError`, `Stream`, `SummaryWriter`, `TensorMap1D`, `TensorMap2D`, `ValueError`, `WeatherGrid`, `ZoneInfo`, `abs`
- 戻り値の要点: `0 / torch.clamp(pv, max=pv_power_limit_w) if pv_power_limit_w > 0.0 else pv / (t_sec + duration, soc, tb_c) / (cost, soc, min_soc_seen, t_sec, max_current_seen, max_charge_current_seen, max_timing_violation_sec, deadline_violation_sec, trace_rows)`
- 主なローカル代入名: `_`, `accel_limit_kmhps`, `active`, `after`, `air_density_mode`, `air_density_reference_pressure_pa`, `alpha`, `altitude_m`, `ambient_c`, `ambient_grid`, `ambient_temp`, `args`, `aux_grid`, `aux_night_ghi_threshold_wm2`, `average_pack_power`, `batch`, `battery_temp`, `before`, `best_cost`, `best_metrics`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `RuntimeError('CUDA is required; run this script inside a GPU Slurm allocation')`, `RuntimeError('checkpoint control dimension does not match the requested profile')`, `RuntimeError('checkpoint input signature does not match the profile, route, weather, stop data, search settings, or proposer implementation; use a new output directory or --no-resume')`, `RuntimeError('checkpoint population does not match the requested population')`, `RuntimeError('search completed without a finite best policy')`, `RuntimeError(f'control stop boundary mismatch: stop={stop_km}, segment_end={stop_end_km}')`, `ValueError('GPU proposer currently requires exactly one repeated daily drive window')`, `ValueError('initial policy must contain s_km and v_kmh')`, `ValueError('initial policy needs at least two finite rows')`, `ValueError('race, integration, and control distances must be positive')`, `ValueError('surrogate trace capture requires exactly one policy')`, `ValueError(f'control stop {stop_km} km is outside the integration horizon')`, `ValueError(f'multiple control stops share segment {stop_index}')`
- 制御構造の規模: 条件分岐 49、ループ 6、try 1

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `args` に `parse_args()` の結果を代入する。
2. `args.cuda_graph` に `resolve_cuda_graph_enabled(args.cuda_graph, args.integration_ds_km)` の結果を代入する。
3. 条件 `not torch.cuda.is_available()` を判定し、真なら内部処理を行う。
4. `RuntimeError('CUDA is required; run this script inside a GPU Slurm allocation')` を送出する。

5. device に torch.device('cuda') の結果を代入する。
6. torch.manual_seed(...) を実行する。
7. profile に args.profile.resolve() の結果を代入する。
8. cfg に yaml.safe_load(profile.read_text(encoding='utf-8')) or {} の結果を代入する。
9. model に cfg['model'] の結果を代入する。
10. mpc に cfg['mpc'] の結果を代入する。
11. paths に cfg['paths'] の結果を代入する。
12. simulation に cfg['simulation'] の結果を代入する。
13. start_utc に iso_utc(simulation['start_utc']) の結果を代入する。
14. race_km に float(mpc['race_km']) の結果を代入する。
15. ds_km に float(args.integration_ds_km) の結果を代入する。
16. ctrl_ds_km に float(args.control_ds_km) の結果を代入する。
17. 条件 race_km <= 0.0 or ds_km <= 0.0 or ctrl_ds_km <= 0.0 を判定し、真なら内部処理を行う。
18. ValueError('race, integration, and control distances must be positive') を送出する。
19. stop_path に resolve(profile, paths['stop_yaml']) の結果を代入する。
20. stops_cfg に yaml.safe_load(stop_path.read_text(encoding='utf-8')) or {} の結果を代入する。
21. stops に [] の結果を代入する。
22. stops_cfg.get('stops', []) を順に走査し、各要素を row に入れて処理する。
23. stop_km に float(row['s_km']) の結果を代入する。
24. 条件 not 0.0 < stop_km < race_km を判定し、真なら内部処理を行う。
25. Continue 文を実行する。
26. open_utc に str(row.get('window_open_utc', '')) or '' の結果を代入する。
27. close_utc に str(row.get('window_close_utc', '')) or '' の結果を代入する。
28. stops.append(...) を実行する。
29. stops.sort(...) を実行する。
30. finish_cfg に stops_cfg.get('finish', {}) or {} の結果を代入する。
31. deadline_utc に str(simulation.get('race_deadline_utc', finish_cfg.get('window_close_utc', '')) or '') の結果を代入する。
32. deadline_sec に float((iso_utc(deadline_utc) - start_utc).total_seconds()) if deadline_utc else float('inf') の結果を代入する。
33. (s_segments, ds_segments, boundaries) に build_distance_segments(race_km, ds_km, [row['s_km'] for row in stops]) の結果を代入する。
34. ctrl_s に np.arange(0.0, race_km, ctrl_ds_km, dtype=np.float32) の結果を代入する。
35. ctrl_s に np.append(ctrl_s, np.float32(race_km)) の結果を代入する。
36. n_ctrl に len(ctrl_s) の結果を代入する。

37. route に `pd.read_csv(resolve(profile, paths['route_profile_csv']))` の結果を代入する。
38. slope に `average_profile_segments(route, s_segments, ds_segments, 'slope_pct', 0.0).astype(np.float32)` の結果を代入する。
39. elevation に `average_profile_segments(route, s_segments, ds_segments, 'elev_m', 0.0).astype(np.float32)` の結果を代入する。
40. speed_profile_path に `resolve(profile, paths['speed_profile_csv'])` の結果を代入する。
41. speed_profile に `pd.read_csv(speed_profile_path)` の結果を代入する。
42. speed_s に `pd.to_numeric(speed_profile.get('dist_km', speed_profile.get('s_km')), errors='coerce').to_numpy(dtype=float)` の結果を代入する。
43. speed_limit_values に `pd.to_numeric(speed_profile['v_max_kmh'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
44. speed_limits に `np.interp(s_segments, speed_s, speed_limit_values).astype(np.float32)` の結果を代入する。
45. weather に `WeatherGrid(resolve(profile, paths['forecast_csv']), iso_utc(simulation['start_utc']), device)` の結果を代入する。
46. stationary_weather_step_sec に `max(1.0, float(mpc.get('stationary_prediction_step_sec', 60.0)))` の結果を代入する。
47. weather_refinement に `weather.refine_time_grid(stationary_weather_step_sec)` の結果を代入する。
48. route_path に `resolve(profile, paths['route_profile_csv'])` の結果を代入する。
49. weather_path に `resolve(profile, paths['forecast_csv'])` の結果を代入する。
50. schedule_path に `resolve(profile, paths['drive_schedule_yaml'])` の結果を代入する。
51. drive_path に `resolve(profile, paths['drive_eff_map'])` の結果を代入する。
52. regen_path に `resolve(profile, paths['regen_eff_map'])` の結果を代入する。
53. drive_eco_path に `resolve(profile, paths.get('drive_map_eco', paths['drive_eff_map']))` の結果を代入する。
54. drive_power_path に `resolve(profile, paths.get('drive_map_power', paths['drive_eff_map']))` の結果を代入する。
55. regen_eco_path に `resolve(profile, paths.get('regen_map_eco', paths['regen_eff_map']))` の結果を代入する。
56. regen_power_path に `resolve(profile, paths.get('regen_map_power', paths['regen_eff_map']))` の結果を代入する。
57. panel_path に `resolve(profile, paths['panel_eff_map'])` の結果を代入する。
58. mppt_path に `resolve(profile, paths['mppt_eff_map'])` の結果を代入する。
59. rint_path に `resolve(profile, paths['rint_map'])` の結果を代入する。
60. ocv_path に `resolve(profile, paths['ocv_soc_map'])` の結果を代入する。
61. drive_default_map に `TensorMap2D(drive_path, device)` の結果を代入する。
62. regen_default_map に `TensorMap2D(regen_path, device)` の結果を代入する。
63. drive_eco_map に `TensorMap2D(drive_eco_path, device)` の結果を代入する。
64. drive_power_map に `TensorMap2D(drive_power_path, device)` の結果を代入する。
65. regen_eco_map に `TensorMap2D(regen_eco_path, device)` の結果を代入する。

66. `regen_power_map` に `TensorMap2D(regen_power_path, device)` の結果を代入する。
67. `panel_map` に `TensorMap2D(panel_path, device)` の結果を代入する。
68. `mppt_map` に `TensorMap2D(mppt_path, device)` の結果を代入する。
69. `rint_map` に `TensorMap2D(rint_path, device)` の結果を代入する。
70. `ocv_map` に `TensorMap1D(ocv_path, device)` の結果を代入する。
71. `stop_by_segment` に `{}` の結果を代入する。
72. `stops` を順に走査し、各要素を `stop` に入れて処理する。
73. `stop_km` に `float(stop['s_km'])` の結果を代入する。
74. `stop_index` に `int(np.searchsorted(boundaries, stop_km, side='left') - 1)` の結果を代入する。
75. 条件 `stop_index < 0 or stop_index >= len(s_segments)` を判定し、真なら内部処理を行う。
76. `ValueError(f'control stop {stop_km} km is outside the integration horizon')` を送出する。
77. `stop_end_km` に `float(s_segments[stop_index] + ds_segments[stop_index])` の結果を代入する。
78. 条件 `not math.isclose(stop_end_km, stop_km, rel_tol=0.0, abs_tol=0.0002)` を判定し、真なら内部処理を行う。
79. `RuntimeError(f'control stop boundary mismatch: stop={stop_km}, segment_end={stop_end_km}')` を送出する。
80. 条件 `stop_index in stop_by_segment` を判定し、真なら内部処理を行う。

代表コード断片

```
def main() -> int:
    args = parse_args()
    # Existing queued campaigns cannot receive new Slurm environment flags.
    # Keep large fine-grid graphs eager while auto-accelerating the measured
    # coarse-grid case after its deployment benchmark passes.
    args.cuda_graph = resolve_cuda_graph_enabled(args.cuda_graph, args.integration_ds_km)
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required; run this script inside a GPU Slurm allocation")
    )
    device = torch.device("cuda")
    torch.manual_seed(args.seed)
    profile = args.profile.resolve()
    cfg = yaml.safe_load(profile.read_text(encoding="utf-8")) or {}
    model = cfg["model"]
    mpc = cfg["mpc"]
    paths = cfg["paths"]
    simulation = cfg["simulation"]
    start_utc = iso_utc(simulation["start_utc"])
    race_km = float(mpc["race_km"])
    ds_km = float(args.integration_ds_km)
    ctrl_ds_km = float(args.control_ds_km)
    if race_km <= 0.0 or ds_km <= 0.0 or ctrl_ds_km <= 0.0:
        raise ValueError("race, integration, and control distances must be positive")

    stop_path = resolve(profile, paths["stop_yaml"])
    stops_cfg = yaml.safe_load(stop_path.read_text(encoding="utf-8")) or {}
    stops = []
    for row in stops_cfg.get("stops", []):
        stop_km = float(row["s_km"])
        if not 0.0 < stop_km < race_km:
            continue
```

```

open_utc = str(row.get("window_open_utc", "") or "")
close_utc = str(row.get("window_close_utc", "") or "")
stops.append(
    {
        "s_km": stop_km,
    }
)
...

```

7.29 L912 関数 main.pv_power

- 行範囲: L912–L931
- 所属: main
- 定義: `pv_power(t_sec:torch.Tensor,s_km:float,*,stopped:bool=False)->torch.Tensor`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clamp`, `sample`
- 戻り値の要点: `torch.clamp(pv, max=pv_power_limit_w) if pv_power_limit_w > 0.0 else pv`
- 主なローカル代入名: `eta_mppt`, `eta_panel`, `ghi`, `irradiance`, `poa_drive`, `poa_ideal`, `pv`, `tcell`, `tcell_drive`, `tcell_ideal`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `ghi` に `weather.sample('GHI', t_sec, s_km)` の結果を代入する。
2. `poa_drive` に `weather.sample('POA_drive', t_sec, s_km)` if 'POA_drive' in `weather.values` else `ghi` の結果を代入する。
3. `tcell_drive` に `weather.sample('Tcell_drive_C', t_sec, s_km)` if 'Tcell_drive_C' in `weather.values` else `weather.sample('Tcell_C', t_sec, s_km)` の結果を代入する。
4. `irradiance` に `poa_drive` の結果を代入する。
5. `tcell` に `tcell_drive` の結果を代入する。
6. 条件 `stopped` and 'POA_stop_ideal' in `weather.values` を判定し、真なら内部処理を行う。
7. `poa_ideal` に `weather.sample('POA_stop_ideal', t_sec, s_km)` の結果を代入する。
8. `irradiance` に `poa_drive + stop_tilt_fraction * (poa_ideal - poa_drive)` の結果を代入する。
9. 条件 'Tcell_stop_ideal_C' in `weather.values` を判定し、真なら内部処理を行う。
10. `tcell_ideal` に `weather.sample('Tcell_stop_ideal_C', t_sec, s_km)` の結果を代入する。
11. `tcell` に `tcell_drive + stop_tilt_fraction * (tcell_ideal - tcell_drive)` の結果を代入する。

12. eta_panel に torch.clamp(panel_map.sample(irradiance, tcell) * panel_gain, min=0.0, max=0.35) の結果を代入する。
13. eta_mppt に torch.clamp(mppt_map.sample(irradiance, tcell), min=0.0, max=1.0) の結果を代入する。
14. pv に irradiance * eta_panel * pv_area * eta_mppt の結果を代入する。
15. torch.clamp(pv, max=pv_power_limit_w) if pv_power_limit_w > 0.0 else pv を返す。

代表コード断片

```
def pv_power(t_sec: torch.Tensor, s_km: float, *, stopped: bool = False) -> torch.Tensor:
    ghi = weather.sample("GHI", t_sec, s_km)
    poa_drive = weather.sample("POA_drive", t_sec, s_km) if "POA_drive" in weather.values else ghi
    tcell_drive = (
        weather.sample("Tcell_drive_C", t_sec, s_km)
        if "Tcell_drive_C" in weather.values
        else weather.sample("Tcell_C", t_sec, s_km)
    )
    irradiance = poa_drive
    tcell = tcell_drive
    if stopped and "POA_stop_ideal" in weather.values:
        poa_ideal = weather.sample("POA_stop_ideal", t_sec, s_km)
        irradiance = poa_drive + stop_tilt_fraction * (poa_ideal - poa_drive)
        if "Tcell_stop_ideal_C" in weather.values:
            tcell_ideal = weather.sample("Tcell_stop_ideal_C", t_sec, s_km)
            tcell = tcell_drive + stop_tilt_fraction * (tcell_ideal - tcell_drive)
    eta_panel = torch.clamp(panel_map.sample(irradiance, tcell) * panel_gain, min=0.0, max=0.35)
    eta_mppt = torch.clamp(mppt_map.sample(irradiance, tcell), min=0.0, max=1.0)
    pv = irradiance * eta_panel * pv_area * eta_mppt
    return torch.clamp(pv, max=pv_power_limit_w) if pv_power_limit_w > 0.0 else pv
```

7.30 L944 関数 main.register_stationary_mode

- 行範囲: L944–L1005
- 所属: main
- 定義: register_stationary_mode(prefix:str, tilt_fraction:float) -> None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: append, clamp, float, full_like, register_soc_time_integral, register_time_integral, sample, sqrt, stack, stationary_auxiliary_power_w, where
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: aux_grid, current_matrix, discriminant_matrix, effective_current_layers, eta_grid, mppt_grid, ocv_matrix, pack_grid, poa_grid, pv_grid, r_internal_matrix, resistance_matrix, soc_layer, soc_matrix, tcell_grid
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 4、ループ 1、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. poa_grid に poa_drive_grid の結果を代入する。
2. tcell_grid に tcell_drive_grid の結果を代入する。
3. 条件‘POA_stop_ideal’ in weather.values を判定し、真なら内部処理を行う。
4. poa_grid に $\text{poa_drive_grid} + \text{float}(\text{tilt_fraction}) * (\text{weather.values}[\text{'POA_stop_ideal'}] - \text{poa_drive_grid})$ の結果を代入する。
5. 条件‘Tcell_stop_ideal_C’ in weather.values を判定し、真なら内部処理を行う。
6. tcell_grid に $\text{tcell_drive_grid} + \text{float}(\text{tilt_fraction}) * (\text{weather.values}[\text{'Tcell_stop_ideal_C'}] - \text{tcell_drive_grid})$ の結果を代入する。
7. eta_grid に $\text{torch.clamp}(\text{panel_map.sample}(\text{poa_grid}, \text{tcell_grid}) * \text{panel_gain}, \text{min}=0.0, \text{max}=0.35)$ の結果を代入する。
8. mppt_grid に $\text{torch.clamp}(\text{mppt_map.sample}(\text{poa_grid}, \text{tcell_grid}), \text{min}=0.0, \text{max}=1.0)$ の結果を代入する。
9. pv_grid に $\text{poa_grid} * \text{eta_grid} * \text{pv_area} * \text{mppt_grid}$ の結果を代入する。
10. 条件 $\text{pv_power_limit_w} > 0.0$ を判定し、真なら内部処理を行う。
11. pv_grid に $\text{torch.clamp}(\text{pv_grid}, \text{max}=\text{pv_power_limit_w})$ の結果を代入する。
12. aux_grid に $\text{stationary_auxiliary_power_w}(\text{poa_grid}, \text{day_power_w}=\text{p_aux_stopped}, \text{night_power_w}=\text{p_aux_night}, \text{night_threshold_wm2}=\text{aux_night_ghi_threshold_wm2})$ の結果を代入する。
13. pack_grid に $\text{aux_grid} - \text{pv_grid}$ の結果を代入する。
14. `weather.register_time_integral(...)` を実行する。
15. 条件 $\text{q_nom_ah} \leq 0.0$ を判定し、真なら内部処理を行う。
16. を返す。
17. effective_current_layers に [] の結果を代入する。
18. stationary_soc_grid を順に走査し、各要素を soc_layer に入れて処理する。
19. soc_matrix に $\text{torch.full_like}(\text{pack_grid}, \text{float}(\text{soc_layer}))$ の結果を代入する。
20. ocv_matrix に $\text{ocv_map.sample}(\text{soc_matrix})$ の結果を代入する。
21. r_internal_matrix に $\text{rint_scale} * \text{rint_map.sample}(\text{ambient_grid}, \text{soc_matrix})$ の結果を代入する。
22. resistance_matrix に $\text{torch.clamp}(\text{r_internal_matrix} + \text{r_line_ohm} + \text{r_polarization_ohm}, \text{min}=1\text{e-}05)$ の結果を代入する。
23. discriminant_matrix に $\text{torch.clamp}(\text{ocv_matrix} * \text{ocv_matrix} - 4.0 * \text{resistance_matrix} * \text{pack_grid}, \text{min}=0.0)$ の結果を代入する。
24. current_matrix に $(\text{ocv_matrix} - \text{torch.sqrt}(\text{discriminant_matrix})) / (2.0 * \text{resistance_matrix})$ の結果を代入する。
25. effective_current_layers.append(...) を実行する。
26. `weather.register_soc_time_integral(...)` を実行する。

代表コード断片

```
def register_stationary_mode(prefix: str, tilt_fraction: float) -> None:
    poa_grid = poa_drive_grid
    tcell_grid = tcell_drive_grid
    if "POA_stop_ideal" in weather.values:
        poa_grid = poa_drive_grid + float(tilt_fraction) * (
            weather.values["POA_stop_ideal"] - poa_drive_grid
        )
    if "Tcell_stop_ideal_C" in weather.values:
        tcell_grid = tcell_drive_grid + float(tilt_fraction) * (
            weather.values["Tcell_stop_ideal_C"] - tcell_drive_grid
        )
    eta_grid = torch.clamp(
        panel_map.sample(poa_grid, tcell_grid) * panel_gain,
        min=0.0,
        max=0.35,
    )
    mppt_grid = torch.clamp(
        mppt_map.sample(poa_grid, tcell_grid),
        min=0.0,
        max=1.0,
    )
    pv_grid = poa_grid * eta_grid * pv_area * mppt_grid
    if pv_power_limit_w > 0.0:
        pv_grid = torch.clamp(pv_grid, max=pv_power_limit_w)
    aux_grid = stationary_auxiliary_power_w(
        poa_grid,
        day_power_w=p_aux_stopped,
        night_power_w=p_aux_night,
        night_threshold_wm2=aux_night_ghi_threshold_wm2,
    )
    pack_grid = aux_grid - pv_grid
    weather.register_time_integral(f"{prefix}_pack_energy_j", pack_grid)
    if q_nom_ah <= 0.0:
        return
    effective_current_layers = []
...

```

7.31 L1010 関数 main.apply_wait

- 行範囲: L1010–L1089
- 所属: main
- 定義: `apply_wait(t_sec:torch.Tensor, soc:torch.Tensor, tb_c:torch.Tensor, duration:torch.Tensor, s_km:float, *, control_stop:bool=False)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clamp`, `exp`, `float`, `get`, `sample`, `sample_integral`, `sample_soc_integral`, `sqrt`, `where`, `zeros_like`
- 戻り値の要点: `(t_sec + duration, soc, tb_c)`
- 主なローカル代入名: `active`, `ambient_c`, `average_pack_power`, `charge_end`, `charge_mid`, `charge_mid_updated`, `charge_start`, `current`, `current_key`, `discriminant`, `duration`, `energy_end_j`, `energy_key`, `energy_start_j`, `half_duration`, `internal_loss_w`, `midpoint_t`, `ocv`, `pack_energy_j`, `prefix`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。

- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. duration に `torch.clamp(duration, min=0.0)` の結果を代入する。
2. prefix に `'control_stop' if control_stop else 'camp'` の結果を代入する。
3. energy_key に `f'{prefix}_pack_energy_j'` の結果を代入する。
4. current_key に `f'{prefix}_effective_current_as'` の結果を代入する。
5. energy_start_j に `weather.sample_integral(energy_key, t_sec, s_km)` の結果を代入する。
6. energy_end_j に `weather.sample_integral(energy_key, t_sec + duration, s_km)` の結果を代入する。
7. pack_energy_j に `energy_end_j - energy_start_j` の結果を代入する。
8. active に `duration > 1e-06` の結果を代入する。
9. average_pack_power に `torch.where(active, pack_energy_j / torch.clamp(duration, min=1e-06), torch.zeros_like(duration))` の結果を代入する。
10. midpoint_t に `t_sec + 0.5 * duration` の結果を代入する。
11. ambient_c に `weather.sample('Tamb_C', midpoint_t, s_km)` の結果を代入する。
12. ocv に `ocv_map.sample(soc)` の結果を代入する。
13. r_internal に `rint_scale * rint_map.sample(tb_c, torch.clamp(soc, 0.1, 0.95))` の結果を代入する。
14. resistance に `torch.clamp(r_internal + r_line_ohm + r_polarization_ohm, min=1e-05)` の結果を代入する。
15. discriminant に `torch.clamp(ocv * ocv - 4.0 * resistance * average_pack_power, min=0.0)` の結果を代入する。
16. current に `(ocv - torch.sqrt(discriminant)) / (2.0 * resistance)` の結果を代入する。
17. 条件 `q_nom_ah > 0.0` を判定し、真なら内部処理を行う。
18. half_duration に `0.5 * duration` の結果を代入する。
19. charge_start に `weather.sample_soc_integral(current_key, t_sec, s_km, soc)` の結果を代入する。
20. charge_mid に `weather.sample_soc_integral(current_key, t_sec + half_duration, s_km, soc)` の結果を代入する。
21. soc_mid に `torch.clamp(soc - (charge_mid - charge_start) / (3600.0 * q_nom_ah), min=float(model.get('soc_min', 0.1)), max=soc_max)` の結果を代入する。
22. charge_mid_updated に `weather.sample_soc_integral(current_key, t_sec + half_duration, s_km, soc_mid)` の結果を代入する。
23. charge_end に `weather.sample_soc_integral(current_key, t_sec + duration, s_km, soc_mid)` の結果を代入する。
24. soc_next に `soc_mid - (charge_end - charge_mid_updated) / (3600.0 * q_nom_ah)` の結果を代入する。

25. 上の条件が偽の場合:
26. soc_next に soc - pack_energy_j / (3600.0 * e_nom_wh) の結果を代入する。
27. soc に torch.where(active, torch.clamp(soc_next, min=float(model.get('soc_min', 0.1)), max=soc_max), soc) の結果を代入する。
28. thermal_alpha に 1.0 - torch.exp(-duration / 1800.0) の結果を代入する。
29. internal_loss_w に current * current * torch.clamp(r_internal, min=0.0) の結果を代入する。
30. tb_next に tb_c + thermal_alpha * (ambient_c - tb_c) + internal_loss_w * (1800.0 / 50000.0) * thermal_alpha の結果を代入する。
31. tb_c に torch.where(active, torch.clamp(tb_next, min=temp_min, max=temp_max), tb_c) の結果を代入する。
32. (t_sec + duration, soc, tb_c) を返す。

代表コード断片

```
def apply_wait(
    t_sec: torch.Tensor,
    soc: torch.Tensor,
    tb_c: torch.Tensor,
    duration: torch.Tensor,
    s_km: float,
    *,
    control_stop: bool = False,
):
    duration = torch.clamp(duration, min=0.0)
    prefix = "control_stop" if control_stop else "camp"
    energy_key = f"{prefix}_pack_energy_j"
    current_key = f"{prefix}_effective_current_as"
    energy_start_j = weather.sample_integral(energy_key, t_sec, s_km)
    energy_end_j = weather.sample_integral(energy_key, t_sec + duration, s_km)
    pack_energy_j = energy_end_j - energy_start_j
    active = duration > 1.0e-6
    average_pack_power = torch.where(
        active,
        pack_energy_j / torch.clamp(duration, min=1.0e-6),
        torch.zeros_like(duration),
    )
    midpoint_t = t_sec + 0.5 * duration
    ambient_c = weather.sample("Tamb_C", midpoint_t, s_km)
    ocv = ocv_map.sample(soc)
    r_internal = rint_scale * rint_map.sample(tb_c, torch.clamp(soc, 0.1, 0.95))
    resistance = torch.clamp(r_internal + r_line_ohm + r_polarization_ohm, min=1.0e-5)
    discriminant = torch.clamp(
        ocv * ocv - 4.0 * resistance * average_pack_power,
        min=0.0,
    )
    current = (ocv - torch.sqrt(discriminant)) / (2.0 * resistance)
    if q_nom_ah > 0.0:
        half_duration = 0.5 * duration
        charge_start = weather.sample_soc_integral(
...

```

7.32 L1091 関数 main.evaluate

- 行範囲: L1091–L1425
- 所属: main

- 定義: `evaluate(policy:torch.Tensor,*,capture_trace:bool=False)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `append`, `append_wait_trace`, `apply_wait`, `atan`, `clamp`, `clone`, `cos`, `float`, `floor`, `full`, `full_like`
- 戻り値の要点: (`cost`, `soc`, `min_soc_seen`, `t_sec`, `max_current_seen`, `max_charge_current_seen`, `max_timing_violation_sec`, `deadline_violation_sec`, `trace_rows`)
- 主なローカル代入名: `after`, `alpha`, `altitude_m`, `ambient_temp`, `batch`, `battery_temp`, `before`, `charge_current_violation`, `charge_factor`, `cost`, `ctrl_idx`, `ctrl_pos`, `current`, `current_violation`, `day_end`, `day_index`, `day_start`, `deadline_violation_sec`, `discriminant`, `drive_eff`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('surrogate trace capture requires exactly one policy')`
- 制御構造の規模: 条件分岐 13、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `batch` に `policy.shape[0]` の結果を代入する。
2. 条件 `capture_trace and batch != 1` を判定し、真なら内部処理を行う。
3. `ValueError('surrogate trace capture requires exactly one policy')` を送出する。
4. `trace_rows` に `[]` を代入する。
5. 関数 `append_wait_trace` を定義する。
6. `t_sec` に `torch.zeros(batch, device=device)` の結果を代入する。
7. `soc` に `torch.full((batch,), soc0, device=device)` の結果を代入する。
8. `tb_c` に `torch.full((batch,), tb0, device=device)` の結果を代入する。
9. `initial_speed_ms` に `float(simulation.get('v0_kmh', simulation.get('gps_init_speed_kmh', 0.0))) / 3.6` の結果を代入する。
10. `previous_speed_ms` に `torch.full((batch,), initial_speed_ms, device=device)` の結果を代入する。
11. `min_soc_seen` に `soc.clone()` の結果を代入する。
12. `max_current_seen` に `torch.zeros(batch, device=device)` の結果を代入する。
13. `max_charge_current_seen` に `torch.zeros(batch, device=device)` の結果を代入する。
14. `max_timing_violation_sec` に `torch.zeros(batch, device=device)` の結果を代入する。
15. `range(len(s_segments))` を順に走査し、各要素を `seg_idx` に入れて処理する。
16. `day_index` に `torch.floor((t_sec - window_origin_sec) / 86400.0)` の結果を代入する。
17. `day_start` に `window_origin_sec + day_index * 86400.0` の結果を代入する。

18. `day_end` に `day_start + window_duration_sec` の結果を代入する。
19. `before` に `t_sec < day_start` の結果を代入する。
20. `after` に `t_sec >= day_end` の結果を代入する。
21. `next_start` に `torch.where(before, day_start, day_start + 86400.0)` の結果を代入する。
22. `wait_duration` に `torch.where(before | after, torch.clamp(next_start - t_sec, min=0.0), 0.0)` の結果を代入する。
23. `wait_t_before` に `t_sec.clone() if capture_trace else None` の結果を代入する。
24. `wait_soc_before` に `soc.clone() if capture_trace else None` の結果を代入する。
25. `wait_tb_before` に `tb_c.clone() if capture_trace else None` の結果を代入する。
26. `(t_sec, soc, tb_c)` に `apply_wait(t_sec, soc, tb_c, wait_duration, float(s_segments[seg_idx]))` の結果を代入する。
27. 条件 `capture_trace` を判定し、真なら内部処理を行う。
28. `append_wait_trace(...)` を実行する。
29. `previous_speed_ms` に `torch.where(wait_duration > 1e-06, torch.zeros_like(previous_speed_ms), previous_speed_ms)` の結果を代入する。
30. `min_soc_seen` に `torch.minimum(min_soc_seen, soc)` の結果を代入する。
31. `ctrl_pos` に `float(s_segments[seg_idx] / ctrl_ds_km)` の結果を代入する。
32. `ctrl_idx` に `min(n_ctrl - 2, max(0, int(math.floor(ctrl_pos))))` の結果を代入する。
33. `alpha` に `ctrl_pos - ctrl_idx` の結果を代入する。
34. `speed_target_kmh` に `torch.clamp(policy[:, ctrl_idx] * (1.0 - alpha) + policy[:, ctrl_idx + 1] * alpha, v_min, v_max)` の結果を代入する。
35. `speed_target_kmh` に `torch.minimum(speed_target_kmh, speed_limit_tensor[seg_idx])` の結果を代入する。
36. `target_speed_ms` に `speed_target_kmh / 3.6` の結果を代入する。
37. `(v_ms, segment_end_speed_ms, dt)` に `slew_limited_segment_kinematics(previous_speed_ms, target_speed_ms, float(ds_segments[seg_idx]), accel_limit_kmhps=accel_limit_kmhps, decel_limit_kmhps=decel_limit_kmhps)` の結果を代入する。
38. `drive_t_before` に `t_sec.clone() if capture_trace else None` の結果を代入する。
39. `drive_soc_before` に `soc.clone() if capture_trace else None` の結果を代入する。
40. `drive_tb_before` に `tb_c.clone() if capture_trace else None` の結果を代入する。
41. `drive_previous_speed_ms` に `previous_speed_ms.clone() if capture_trace else None` の結果を代入する。
42. `headwind` に `weather.sample('headwind_ms', t_sec, float(s_segments[seg_idx])) * headwind_scale` の結果を代入する。
43. `ambient_temp` に `weather.sample('Tamb_C', t_sec, float(s_segments[seg_idx]))` の結果を代入する。
44. 条件 `air_density_mode == 'ideal_gas_altitude'` を判定し、真なら内部処理を行う。
45. `altitude_m` に `torch.clamp(elevation_tensor[seg_idx], min=-500.0, max=11000.0)` の結果を代入する。
46. `pressure_ratio` に `torch.clamp(1.0 - 0.0065 * altitude_m / 288.15, min=0.05) ** 5.255877` の結果を代入する。
47. `segment_rho` に `air_density_reference_pressure_pa * pressure_ratio / (287.05 * torch.clamp(ambient_temp + 273.15, min=180.0))` の結果を代入する。

48. 上の条件が偽の場合:
49. `segment_rho` に `rho` の結果を代入する。
50. `theta` に `torch.atan(slope_tensor[seg_idx] * grade_scale / 100.0)` の結果を代入する。
51. `force` に `0.5 * segment_rho * cda * torch.clamp(v_ms + headwind, min=0.0) ** 2` の結果を代入する。
52. `force` に `force + crr * mass * gravity * torch.cos(theta) + mass * gravity * torch.sin(theta)` の結果を代入する。
53. `road_power` に `force * v_ms` の結果を代入する。
54. `inertia_power` に `kinetic_power_w(mass, segment_end_speed_ms, previous_speed_ms, dt)` の結果を代入する。
55. `wheel_power` に `road_power + inertia_power` の結果を代入する。
56. `wheel_power_pos` に `torch.clamp(wheel_power, min=0.0)` の結果を代入する。
57. `wheel_power_neg` に `torch.clamp(-wheel_power, min=0.0)` の結果を代入する。
58. `omega_wheel` に `v_ms / max(wheel_radius, 1e-06)` の結果を代入する。
59. `torque_drive` に `wheel_power_pos / torch.clamp(omega_wheel, min=0.001) / max(gear_ratio, 1e-06)` の結果を代入する。
60. `torque_regen` に `wheel_power_neg / torch.clamp(omega_wheel, min=0.001) / max(gear_ratio, 1e-06)` の結果を代入する。
61. 条件 `drive_mode == 'eco'` を判定し、真なら内部処理を行う。
62. `drive_eff_map_value` に `drive_eco_map.sample(v_ms, torque_drive)` の結果を代入する。
63. `regen_eff_map_value` に `regen_eco_map.sample(v_ms, torque_regen)` の結果を代入する。
64. 上の条件が偽の場合:
65. 条件 `drive_mode == 'power'` を判定し、真なら内部処理を行う。
66. `drive_eff_map_value` に `drive_power_map.sample(v_ms, torque_drive)` の結果を代入する。
67. `regen_eff_map_value` に `regen_power_map.sample(v_ms, torque_regen)` の結果を代入する。
68. 上の条件が偽の場合:
69. 条件 `drive_mode == 'auto'` を判定し、真なら内部処理を行う。
70. `drive_eff` に `torch.clamp(drive_eff_map_value * drive_eff_scale, min=0.55, max=0.99)` の結果を代入する。
71. `regen_eff` に `torch.clamp(regen_eff_map_value * regen_eff_scale, min=0.4, max=0.95)` の結果を代入する。
72. `drive_power` に `wheel_power_pos / torch.clamp(drive_eff * gear_eta * inverter_eta, min=1e-06)` の結果を代入する。
73. `regen_power` に `wheel_power_neg * regen_utilization * regen_eff * gear_eta * inverter_eta` の結果を代入する。
74. `pack_power` に `drive_power - regen_power + p_aux - pv_power(t_sec, float(s_segments[seg_idx]))` の結果を代入する。
75. `battery_temp` に `tb_c` の結果を代入する。
76. `ocv` に `ocv_map.sample(soc)` の結果を代入する。
77. `r_internal` に `rint_scale * rint_map.sample(battery_temp, torch.clamp(soc, 0.1, 0.95))` の結果を代入する。

78. resistance に $r_{\text{internal}} + r_{\text{line_ohm}} + r_{\text{polarization_ohm}}$ の結果を代入する。
79. resistance に `torch.clamp(resistance, min=1e-05)` の結果を代入する。
80. discriminant に `torch.clamp(ocv * ocv - 4.0 * resistance * pack_power, min=0.0)` の結果を代入する。

代表コード断片

```
def evaluate(policy: torch.Tensor, *, capture_trace: bool = False):
    batch = policy.shape[0]
    if capture_trace and batch != 1:
        raise ValueError("surrogate trace capture requires exactly one policy")
    trace_rows: list[dict[str, float | int | str]] = []

    def append_wait_trace(
        label,
        segment_index,
        s_km,
        t_before,
        t_after,
        soc_before,
        soc_after,
        tb_before,
        tb_after,
    ):
        if not capture_trace:
            return
        duration = float((t_after[0] - t_before[0]).item())
        if duration <= 1.0e-6:
            return
        trace_rows.append(
            {
                "phase": str(label),
                "segment_index": int(segment_index),
                "s_km": float(s_km),
                "s_end_km": float(s_km),
                "time_sec": float(t_before[0].item()),
                "time_end_sec": float(t_after[0].item()),
                "dt_sec": duration,
                "soc": float(soc_before[0].item()),
                "soc_end": float(soc_after[0].item()),
                "Tb_C": float(tb_before[0].item()),
                "Tb_end_C": float(tb_after[0].item()),
            }
        )
    ...
```

7.33 L1097 関数 `main.evaluate.append_wait_trace`

- 行範囲: L1097–L1128
- 所属: `main.evaluate`
- 定義: `append_wait_trace(label, segment_index, s_km, t_before, t_after, soc_before, soc_after, tb_before, tb_after)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `float`, `int`, `item`, `str`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `duration`
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `not capture_trace` を判定し、真なら内部処理を行う。
2. を返す。
3. `duration` に `float((t_after[0] - t_before[0]).item())` の結果を代入する。
4. 条件 `duration <= 1e-06` を判定し、真なら内部処理を行う。
5. を返す。
6. `trace_rows.append(...)` を実行する。

代表コード断片

```
def append_wait_trace(
    label,
    segment_index,
    s_km,
    t_before,
    t_after,
    soc_before,
    soc_after,
    tb_before,
    tb_after,
):
    if not capture_trace:
        return
    duration = float((t_after[0] - t_before[0]).item())
    if duration <= 1.0e-6:
        return
    trace_rows.append(
        {
            "phase": str(label),
            "segment_index": int(segment_index),
            "s_km": float(s_km),
            "s_end_km": float(s_km),
            "time_sec": float(t_before[0].item()),
            "time_end_sec": float(t_after[0].item()),
            "dt_sec": duration,
            "soc": float(soc_before[0].item()),
            "soc_end": float(soc_after[0].item()),
            "Tb_C": float(tb_before[0].item()),
            "Tb_end_C": float(tb_after[0].item()),
            "v_kmh": 0.0,
        }
    )
```

7.34 CLI 引数

行	引数
500	--profile
501	--output-dir
502	--integration-ds-km
503	--control-ds-km
504	--population
505	--elite
506	--generations
507	--seed
508	--checkpoint-every
509	--resume
510	--tensorboard-dir
511	--initial-policy
512	--initial-std-kmh
513	--antithetic-sampling
519	--cuda-graph
528	--early-stop-patience
534	--early-stop-min-generations
535	--early-stop-min-delta
536	--early-stop-max-std-kmh
542	--capture-final-surrogate-trace

8 処理の流れ

1. CLI 引数を解釈し、main() から処理を起動する。

9 このファイルの位置づけ

この資料群では、scripts/gpu_upper_policy_search.py を **planning research** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の profile / launch / telemetry と後段の planner / logger / report へ繋がる。